



UNIVERSIDAD DE MÁLAGA



Graduado en Ingeniería Informática

Uso de LLMs para el análisis de vulnerabilidades en Solana “Programs”

Vulnerability analysis of Solana “Programs” using LLMs

Realizado por
Laura De Haro García

Tutorizado por
Isaac Agudo

Departamento
Lenguajes y Ciencias de la Computación

MÁLAGA, septiembre 2025

Abstract

En este estudio, se analiza la posibilidad de empleo de los grandes modelos de lenguaje (*LLMs*) para detectar vulnerabilidades en los contratos inteligentes conocidos como *Programs* escritos dentro de la *Blockchain* Solana. Concretamente, se examinan las siguientes 11 vulnerabilidades: Falta de verificación de *ownership*, verificación de firmas, verificación de *PDA*, inicialización de cuentas, control de privilegios, control del identificador al invocar programas externos, desbordamientos de enteros (*overflow* y *underflow*), uso de *bumps* vulnerables, confusión de cuentas y uso de funciones obsoletas, a lo largo de 126 contratos escritos en Rust. Para ello, se emplean 4 grandes modelos de lenguaje: *GPT Nano 4.1*, *Gemini Flash 2.0*, *DeepSeek V3.1* y *Qwen3 235B A22B*, utilizando 2 combinaciones de *system* y *user prompt* cada uno.

A partir de los datos recogidos, se realiza un análisis estadístico que se fija en dos objetivos: el primero de estos consiste en la evaluación de la capacidad de clasificación binaria de la presencia de las vulnerabilidades individuales en los contratos mediante un análisis de las métricas de la matriz de confusión. El segundo se enfoca en examinar la capacidad de detectar correctamente el número de apariciones de cada vulnerabilidad en un determinado contrato mediante el uso de las clásicas funciones de pérdida (MAE y RMSE), teniendo en cuenta también el sesgo del modelo. De esta manera, este trabajo pretende evaluar tanto las posibilidades como los peligros del uso de los *LLMs* en el ámbito de la arquitectura segura de los *Programs* de Solana.

In this study, the potential use of large language models (LLMs) to detect vulnerabilities in smart contracts known as Programs written on the Solana blockchain is analyzed. Specifically, the following 11 vulnerabilities are examined: lack of ownership verification, signature verification, PDA verification, account initialization, privilege control, identifier control when invoking external programs, integer overflows and underflows, use of vulnerable bumps, account confusion, and the use of deprecated functions, across 126 contracts written in Rust. To this end, 4 large language models are employed: *GPT Nano 4.1*, *Gemini Flash 2.0*, *DeepSeek V3.1* y *Qwen3 235B A22B*, using 2 combinations of system and user prompts each.

Based on the collected data, a statistical analysis is carried out with two objectives: the first consists of evaluating the binary classification capacity for detecting the presence of individual vulnerabilities in the contracts through confusion matrix metrics. The second focuses on assessing the ability to correctly detect the number of occurrences of each vulnerability in a given contract, using classical loss functions (MAE and RMSE), while also considering model bias. In this way, the study aims to evaluate both the possibilities and the risks of using LLMs in the field of secure architecture for Solana Programs.

Keywords: Blockchain, Solana, Program, Smart contract, Prompt, LLM, vulnerability, cibersecurity

Índice

1. Introducción	5
1.1. Motivación	5
1.2. Objetivos	6
1.3. Estructura del documento	6
2. Tecnologías usadas	9
2.1. Blockchain: conceptos clave y principales referencias	9
2.2. Modelo y funcionamiento de la red Solana	12
2.3. Lenguaje de programación Rust aplicado a <i>Programs</i>	15
2.4. Concepto de contratos inteligentes y su papel en la Web3:	16
2.5. Modelos Extensos de Lenguaje (LLMs)	17
2.6. Otros	20
3. Desarrollo	21
3.1. Fase 1: Análisis e investigación de vulnerabilidades en <i>Programs</i> de Solana. . .	22
3.1.1. <i>Owner Check</i>	22
3.1.2. <i>Signer Check</i>	22
3.1.3. <i>PDA Check</i>	23
3.1.4. <i>Arithmetic Overflow Check</i>	23
3.1.5. <i>Arithmetic Underflow Check</i>	24
3.1.6. <i>Account Initialization Check</i>	24
3.1.7. <i>Admin Privileges Check</i>	25
3.1.8. <i>Account Confusion Check</i>	25
3.1.9. <i>Bump</i> vulnerable	26
3.1.10. CPI ID Check	26
3.1.11. <i>Wormhole Attack</i> : Caso real	26
3.2. Fase 2: Diseño de <i>Programs</i> vulnerables	30
3.2.1. Programa <i>Wallet</i>	30

3.2.2.	Programa <i>Voting</i>	32
3.3.	Fase 3: Modelos seleccionados, recopilación de datos y diseño de <i>Prompts</i> para la ayuda en la búsqueda de vulnerabilidades	39
3.3.1.	Modelos extensos de lenguaje (<i>LLM</i>)	39
3.3.2.	Generación de datos	43
3.3.3.	Criterios de evaluación y metodología del análisis	49
3.4.	Fase 4: Análisis de los resultados	51
3.4.1.	Capacidad de detectar la presencia de vulnerabilidades	51
3.4.2.	Capacidad de detección del número correcto de vulnerabilidades	59
3.4.3.	Detecciones de la función deprecada <i>load_function_at</i> y el uso de la cuenta <i>sysvar instructions</i>	67
4.	Conclusiones y Líneas Futuras	69

1

Introducción

1.1. Motivación

En un mundo dónde se aboga cada vez más por aumentar la transparencia en la red y la descentralización de muchos servicios, entra en juego con fuerza la arquitectura *Blockchain*, la cual cumple con estos principios cada vez más deseados por los usuarios de internet. Podemos ver claramente cómo con el paso de los años esta estructura es más presente en el mundo virtual y es por eso que debemos explorar las posibles vulnerabilidades y ataques que puede sufrir, pues al fin y al cabo sigue siendo una tecnología relativamente nueva y muy susceptible a ataques. Uno de los componentes principales de esta estructura son los conocidos *smart contracts* o contratos inteligentes, cuya función principal es ejecutar automáticamente transacciones y acuerdos que se lleven a cabo en *Blockchain* sin intervención de terceros. Estos contratos inteligentes están escritos en diferentes lenguajes de programación, dependiendo de la *Blockchain* utilizada, y son fácilmente explotables si el programador no conoce bien el lenguaje y sus vulnerabilidades. Este problema es crítico de abarcar ya que cada vez más usuarios y grandes empresas utilizan estas tecnologías para realizar sus trámites de forma segura y transparente. Es más, se especula que en un futuro podrían llegar a sustituir completamente el sistema legal tal y como lo conocemos y automatizarlo completamente.

Este estudio se enfocara en la *Blockchain* Solana, una plataforma relativamente nueva que ha ganado una gran popularidad en los últimos años. Se investigarán las características de la tecnología Solana y las prácticas seguras para el desarrollo de *smart contracts*, los cuales se conocen como *Programs* en la red, indagando en distintos tipos de vulnerabilidades, desde las mas básicas y fundamentales, a problemas lógicos y abstractos, como a vulnerabilidades presentes en ataques reales.

Se usara Rust, principalmente por ser el lenguaje de programación usado por Solana

para crear *Programs*, pero también cabe resaltar la creciente demanda y popularidad que está ganando, actualmente se encuentra entre los 20 primeros en el índice TIOBE [1], un indicador que mide la popularidad de los lenguajes de programación. Pero además, grandes empresas como son Microsoft, Google y Amazon ya comienzan a implantarlo en proyectos relacionados con el alto rendimiento y la seguridad, cualidades intrínsecas de Rust. Incluso Linux ha comenzado a usar Rust para el desarrollo del núcleo de su sistema operativo.

Una vez creado los *Programs* vulnerables, a los que llamaremos contratos o programas a partir de ahora, se usarán distintos modelos de lenguaje (*LLM*) del mercado para evaluar su nivel de eficiencia a la hora de la búsqueda de vulnerabilidades y problemas del código.

1.2. Objetivos

Entre los objetivos comprendidos en este trabajo se encuentran:

- Conocer la sintaxis y la estructura del lenguaje de programación Rust.
- Estudiar la arquitectura de Solana y compararla con otras tecnologías *Blockchain*, destacando sus diferencias.
- Analizar en detalle los posibles problemas y vulnerabilidades presentes que se pueden encontrar en el código de un *Program* (contrato inteligente).
- Investigar casos reales, analizar los problemas que hubo y su correspondiente mitigación.
- Diseñar *prompts* para distintos modelos *LLM* del mercado, evaluar su utilidad en la detección de vulnerabilidades de código y comparar sus respuestas.

1.3. Estructura del documento

El documento se estructura en varios apartados. En primer lugar, la introducción, donde se presentan la motivación y los objetivos del trabajo. A continuación, se detallan las tecnologías empleadas, explicando los conceptos y definiciones necesarias para comprenderlas. En este apartado se abordan temas como *Blockchain*, diferentes tipos, modelos de lenguaje extenso, las características destacadas de Solana y el concepto de contratos inteligentes, entre otros.

El desarrollo, describe con detalle el proceso seguido durante la investigación. Se incluyen la definición de las vulnerabilidades estudiadas, el proceso de creación de programas vulnerables, así como la recopilación y el análisis de las pruebas realizadas.

Finalmente, en la conclusión y líneas futuras, se presentan los resultados obtenidos y se plantean posibles continuaciones y mejoras para futuros estudios.

Tecnologías usadas

2.1. Blockchain: conceptos clave y principales referencias

De una manera muy técnica, el NIST en su artículo [2, p. 6] define la tecnología *blockchain* de la siguiente forma:

Las blockchains son libros contables digitales a prueba de manipulación y resistentes a alteraciones, implementados de manera distribuida (es decir, sin un repositorio central) y, por lo general, sin una autoridad central como un banco, una empresa o un gobierno. En su nivel más básico, permiten que una comunidad de usuarios registre transacciones en un libro compartido dentro de esa misma comunidad, de modo que, bajo el funcionamiento normal de la red blockchain, ninguna transacción pueda ser modificada una vez publicada.

En términos más simples, esta definición resalta que la *blockchain* constituye un mecanismo para realizar transacciones y operaciones de forma transparente y descentralizada. Aunque en un principio el enfoque estuvo en el ámbito de las transferencias monetarias, la tecnología ha evolucionado mucho más allá de esa primera aplicación.

El artículo [2, p. 6] continúa relatando que a partir de esta idea, se combinó con otras tecnologías y conceptos informáticos para dar origen a lo que conocemos hoy como *criptomonedas*, un activo digital basado en mecanismos criptográficos que no depende de ninguna autoridad central. Fue entonces cuando se creó **Bitcoin**, la primera criptomoneda basada en *blockchain*, lanzada en 2008 por el creador o grupo de creadores bajo el seudónimo de Satoshi Nakamoto.

Esta moneda digital fue diseñada para funcionar de manera *peer-to-peer (P2P)*, es decir, sin la necesidad de que un tercero participe en el proceso de transferencia. Para lograr este objetivo, el sistema opera en base a una **cadena de firmas digitales**, concepto definido en [3, p. 2]:

Cada propietario transfiere la moneda al siguiente firmando digitalmente un hash de la transacción anterior junto con la clave pública del siguiente propietario, y añadiendo estos elementos al final de la moneda. De esta manera, el receptor puede verificar las firmas para comprobar la cadena de propiedad.

Sin embargo, esto por si solo no prohíbe que un propietario puede gastar la misma moneda dos veces (*double spending*). Para solucionar este problema, se implementa un sistema de prueba de trabajo también llamado *Proof of Work (PoW)* explicado en [3, p. 3]:

... consiste en buscar un valor que, al ser procesado mediante una función hash (por ejemplo, SHA-256), produzca un hash que comience con un número determinado de bits en cero. El trabajo promedio requerido crece de forma exponencial con respecto al número de bits en cero exigidos, mientras que la verificación es sencilla, ya que solo requiere ejecutar un único cálculo hash.

Su funcionamiento es el siguiente, cada nuevo bloque añadido a la cadena debe calcular un nuevo *hash* a partir de: el *hash* del bloque anterior, las instrucciones de la transacción y un *nonce*, el cuál va incrementando su valor hasta dar con un *hash* que cumpla con la condición anterior. Finalmente, [3, p. 3] asegura que una vez invertido el esfuerzo computacional para cumplir con la prueba de trabajo, un nuevo bloque añadido a la *blockchain* no podrá ser rehecho sin modificar todos los que le preceden gracias a la firma de los *hashes*.

Esta es la base de la moneda digital Bitcoin, cuya aplicación hizo que la tecnología *blockchain* ganará gran popularidad como sistema de pagos y reserva de valor. Sin embargo, este sistema comenzó a mostrar limitaciones y no era suficiente para satisfacer los nuevos requerimientos que empezaba a demandar la tecnología.

En 2013 Vitalik Buterin publicó el artículo [4] donde introducía **Ethereum**, una plataforma capaz de ejecutar contratos inteligentes y aplicaciones descentralizadas (*dApps*), un concepto revolucionario para la aplicación práctica de tecnologías *blockchain*.

Ethereum se diferencia de una manera clave de Bitcoin en el manejo de los estados de la red. Los estados de Bitcoin consisten en el número total de monedas digitales que existen y no se han gastado aún, llamadas *unspent transaction outputs* o *UTXO* según [4, p. 5]. Cada una de ellas tiene un valor asociado (0.5 BTC, 1 BTC ...) y un propietario que se identifica como la clave pública del usuario. El saldo total de una persona en la red es la suma de todos los *UTXO* que posee y para realizar transacciones se utiliza *UTXO* en las entradas y una vez realizada, si sobran, se devuelven al propietario.

Sin embargo, Ethereum en [4, p. 13] reemplaza este enfoque por un sistema más sencillo. En lugar de que el estado de la red esté compuesto por todas las *UTXO*, está formado por cuentas de Ethereum, las cuales contienen cuatro campos: un saldo global, un *nonce*, un código de contrato y un almacenamiento.

Esto fue posible gracias a la incorporación de un lenguaje de programación Turing-completo, que permite a los desarrolladores definir su propia lógica dentro de la *blockchain*, algo inexistente en Bitcoin con su limitado Bitcoin Scripting.

Algo nuevo que se introdujo gracias a la implementación de contratos fue que, además de las cuentas creadas por agentes externos, ahora también existían cuentas generadas por los propios contratos dentro de la *blockchain*.

Finalmente, para poder implementar contratos inteligentes en la red, Ethereum explica en [4, p. 14] la creación de un sistema de tarifas para las transacciones basado en el consumo real de recursos computacionales. De esta manera evita que código de los contratos se ejecutara de manera descontrolada. Las variables que controlan este aspecto son *STARTGAS*, que establece un límite de los pasos computacionales ejecutados por transacción, y *GASPRICE*, donde el remitente paga por cada paso computacional.

De esta forma, se abrieron las puertas a nuevos ecosistemas como las finanzas descentralizadas (*DeFi*) y los activos digitales únicos (*NFTs*), ampliando significativamente el alcance y las posibilidades de la tecnología *blockchain*. El propósito de Ethereum es servir como una plataforma flexible para crear aplicaciones descentralizadas, capaces de establecer reglas de propiedad, transacciones y estados. Gracias a ello, es posible desplegar protocolos complejos y contratos inteligentes que gestionan valor bajo condiciones específicas, superando así las limitaciones de Bitcoin.

Por su parte Solana, lanzada en 2020 por Anatoly Yakovenko con su artículo [5], comparte con Ethereum la capacidad de desplegar contratos inteligentes, llamados *Programs*, y de soportar el desarrollo de aplicaciones descentralizadas y diversos servicios. No obstante, fue concebida con un objetivo concreto, resolver una de las principales limitaciones de Ethereum: la velocidad y el coste de las transacciones, ofreciendo una infraestructura mucho más rápida y con tarifas significativamente más bajas.

Gracias a su diseño, Solana se caracteriza por su alto rendimiento y gran velocidad de procesamiento, alcanzando miles de transacciones por segundo con comisiones mínimas. Esto es posible gracias a su novedoso mecanismo de consenso, llamado *Proof of History (PoH)*

explicado en [5, p. 3], que combina la prueba de participación (*Proof of Stake*) con un sistema de relojes criptográficos. El mecanismo *Proof of History* permite ordenar las transacciones de manera muy eficiente sin necesidad de que todos los nodos se comuniquen constantemente entre sí, reduciendo la latencia y aumentando el rendimiento. En la práctica, esto hace que Solana pueda manejar aplicaciones de gran escala, como intercambios descentralizados, videojuegos en blockchain o sistemas financieros complejos, sin que los costos se disparen como ocurre en otras redes.

En resumen, Solana ofrece capacidades similares a Ethereum en cuanto a contratos inteligentes y aplicaciones descentralizadas, pero con un enfoque en la velocidad, escalabilidad y bajos costos, lo que la convierte en una alternativa atractiva para proyectos que requieren gran capacidad de procesamiento. Sin embargo, debido a su arquitectura monolítica, sacrifica parte de la descentralización en favor de la eficiencia y el bajo coste, mientras que Ethereum, gracias a su arquitectura modular y por capas, logra un mayor equilibrio entre seguridad, descentralización y escalabilidad.

Característica	BITCOIN	ETHEREUM	SOLANA
Token nativo	BTC	ETH	SOL
Velocidad aprox (tps)	7	15-30 (red principal)	65000
Arquitectura	Monolítica	Modular + Ethereum Virtual Machine (EVM)	Monolítica
Lenguaje de programación	Script	Solidity	Rust
Mecanismo de consenso	Proof of Work (PoW)	Actualmente Proof of Stake (PoS)	PoS + Proof of History (PoH)

Cuadro 1: Diferencias clave entre principales tecnologías *blockchain*

2.2. Modelo y funcionamiento de la red Solana

La característica principal de la arquitectura de Solana, que la diferencia de otras *blockchains* como Ethereum, es que los programas no tienen estado (*stateless*), es decir, no almacenan información de manera interna, sino que únicamente ejecutan código. El estado de los programas se gestiona a través de las cuentas, las cuales se utilizan en Solana para almacenar e interactuar con todos los elementos de la red. Este diseño permite una mayor paralelización en el procesamiento de transacciones, lo que a su vez mejora significativamente la escalabilidad.

La actualización de cuentas y programas en Solana se realiza mediante instrucciones enviadas a través de transacciones, y la capacidad de modificar un programa recae exclusivamente en su autoridad. Es decir, todas las cuentas creadas en Solana tienen un campo llamado *owner*,

donde se almacena el ID o clave pública del programa que la creó, siendo este a la vez el único programa autorizado para alterar los datos contenidos en la cuenta.

Como bien se explica en su página oficial [6], en Solana todo lo que se crea es una cuenta, desde aquellas que almacenan datos de un usuario o estructura, hasta las que contienen el código de un programa. Todas las cuentas comparten la misma estructura básica, lo único que varía es el contenido almacenado en el campo datos.

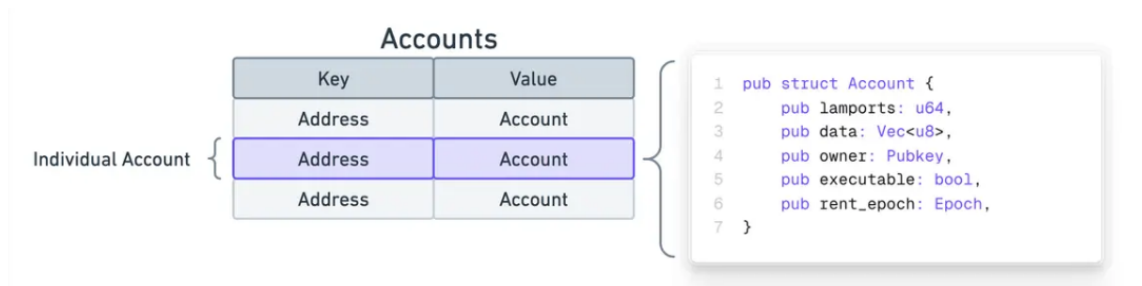


Figura 1: Modelo de estructura de cuenta en Solana. Imagen obtenida de [6]

Además, todas las cuentas en Solana incluyen variables que indican si su contenido es ejecutable o no, es decir, si se trata de un programa o simplemente de una cuenta que almacena un estado. Cada cuenta también almacena su balance en *lamports*, que es la unidad mínima de SOL (1 SOL = 1.000.000.000 lamports).

El sistema de Solana también diferencia entre cuentas externas e internas de la red. Todas las cuentas poseídas por una entidad externa, tienen asociado una clave pública y una privada, siendo esta última la que permite firmar y verificar transacciones. Este par de claves equivale a un punto en la curva **Ed25519** (criptografía de curva elíptica).

Sin embargo, las cuentas generadas por los programas internos de la red, no generan este par de llaves, solo poseen una clave pública, por lo que para generar esa clave debe encontrarse fuera de la curva Ed25519. A este tipo de cuenta se les llama *Program-Derived Addresses* o PDA, y según el sitio oficial [7]:

Las PDAs son direcciones derivadas determinísticamente utilizando una combinación de seeds predefinidos, un bump seed y el ID de un programa.

Las *seeds* pueden ser palabras, claves públicas, o cualquier otro tipo de variable. Los *bumps* por otra parte, son un número entre 0 - 255 que aseguran que la dirección derivada se encuentra fuera de la curva criptográfica. Estas cuentas solo pueden ser creadas a a partir de un programa

Program y para firmar transacciones, solamente el programa que las derivó puede firmar en su nombre.

Dentro del entorno de ejecución de Solana, existen diversas cuentas que cumplen funciones cruciales para el funcionamiento de la *blockchain*. Algunas de ellas se mencionan a continuación por su relevancia para este trabajo, aunque existen muchas más en la red y se pueden consultar en [8].

- **System Program:** Cuenta de programa encargada de inicializar y crear todas las cuentas nuevas en la *blockchain*. Sus funciones principales incluyen crear cuentas, transferir SOL entre cuentas y asignar programas a las cuentas mediante el campo *owner*.

- **Sysvar Accounts:** Conjunto de cuentas de datos que almacenan variables globales de la *blockchain* y que pueden ser útiles para programas o transacciones. Entre ellas se encuentran cuentas como *Clock* la cual mantiene información sobre el tiempo y *epochs*.

- **SPL Token Program:** Cuenta de programa encargada de la creación y gestión de *tokens* en Solana. Permite crear *tokens* personalizados, gestionar balances, realizar transferencias y manejar permisos asociados a las cuentas de los *tokens*.

- **BPF Loader:** Cuenta de programa encargada de cargar y ejecutar programas compilados en *BPF (Berkeley Packet Filter)*, el formato que usa Solana para sus *Programs*. Permite que los programas escritos en Rust se desplieguen y actualicen en la *blockchain*.

Para interactuar con un programa en Solana, se debe crear una transacción, dentro de la cual se incluyen instrucciones, que pueden ser más de una. De esta manera, es posible ejecutar varias llamadas a programas diferentes dentro de una misma transacción.

Los parámetros que siempre se deben enviar a los programas en Solana son las cuentas. Por lo general, todos los programas necesitan una lista de cuentas para ejecutar una función específica. Los demás parámetros adicionales, como strings, enteros u otras variables, se incluyen en el campo llamado *instruction data* dentro de la transacción. El siguiente código es un ejemplo de una transacción sencilla en la que se envían las cuentas necesarias para realizar la creación de una nueva cuenta para un programa en específico.

Listing 1: Código en JS para la creación de una transacción

```
1 // Añadir la información necesaria a los datos de la instrucción
2 //(depende de los parametros que necesite la función)
3 const instructionData = Buffer.from([0])
4 // Crear la instrucción
```

```

5 const instruction = new TransactionInstruction({
6   keys: [
7     { pubkey: userKeypair.publicKey, isSigner: true,
8       isWritable: true },
9     { pubkey: walletPda, isSigner: false, isWritable: true },
10    { pubkey: SystemProgram.programId, isSigner: false,
11      isWritable: false },
12  ],
13  programId: PROGRAM_ID,
14  data: instructionData,
15 });
16 // Crear y enviar la transacción
17 const tx = new Transaction().add(instruction);

```

Listing 1: Código en JS para la creación de una transacción

Finalmente, cabe resaltar que la red de Solana cuenta con cuatro entornos principales:

Mainnet Beta, Testnet, Devnet y Localnet.

Dado que este trabajo tiene un carácter meramente exploratorio, se utilizará la red Devnet, diseñada para que los desarrolladores puedan probar sus programas en Solana sin coste alguno.

2.3. Lenguaje de programación Rust aplicado a *Programs*

Tal como se ve en la documentación oficial de Rust [9], se trata de un lenguaje de programación lanzado en 2010 *multiparadigma* y de propósito *general* que combina la eficiencia de los lenguajes de bajo nivel con mejoras significativas en seguridad y flexibilidad para los desarrolladores. Permite crear aplicaciones de diversos tipos, desde servicios web y APIs hasta herramientas de línea de comandos y software de sistema.

Rust destaca por su alto rendimiento, ya que compila directamente a código máquina, y por su seguridad de memoria en tiempo de compilación, gracias a su sistema de *ownership* y *borrowing*, que previene errores comunes como punteros nulos o fugas de memoria. Además, facilita la concurrencia segura, evitando *condiciones de carrera* al desarrollar programas *multihilo*.

En conjunto, Rust se presenta como una alternativa moderna a lenguajes tradicionales de bajo nivel como C y C++, ofreciendo un balance entre eficiencia, seguridad y facilidad de desarrollo.

Rust fue elegido específicamente por Solana como su lenguaje de programación oficial, ya que se alinea perfectamente con los intereses y fortalezas clave de la plataforma: velocidad y eficiencia, que son objetivos centrales de Solana. Además, Rust ofrece seguridad intrínseca y un modelo de concurrencia que permite al entorno de ejecución de Solana manejar múltiples transacciones simultáneamente con una baja probabilidad de errores, mejorando así la escalabilidad y la eficiencia general de la red.

A partir de esta elección, Solana ha desarrollado paquetes específicos para integrar Rust con su ecosistema, incluyendo:

- **Solana Program Library:** para el desarrollo de programas en la **blockchain** [10].
- **Solana SDK:** para interacción desde el cliente [11].
- **Solana CLI:** herramienta de línea de comandos para el desarrollo en Solana [12].

Además, la comunidad ha creado numerosos paquetes configurables mediante Cargo, facilitando la integración y el desarrollo. Aunque es posible interactuar con la red Solana desde otros lenguajes, como JavaScript a través de *@solana/web3.js* [13] que además se usará en el desarrollo del proyecto para comunicarse con la red.

También existe un *framework* para principiantes llamado *Anchor* [14], que permite reducir código repetitivo, automatizar configuraciones y realizar verificaciones de posibles vulnerabilidades de manera automática. Sin embargo, el uso de Rust nativo ofrece un control más detallado del código, lo que lo hace más adecuado para el análisis profundo de vulnerabilidades, que es el objetivo principal de este trabajo.

2.4. Concepto de contratos inteligentes y su papel en la Web3:

Con la llegada de las tecnologías *blockchain* ha surgido también la idea de la próxima generación de la web, conocida como *Web3*. Según [15, p. 5], esta se caracteriza por potenciar la seguridad, transparencia y descentralización en el uso de Internet. Esta nueva era busca superar el modelo centralizado de grandes empresas que monopolizan el acceso a recursos e información y permite en ámbitos tan esenciales como las transacciones monetarias, eliminar la dependencia de terceros, permitiendo que las interacciones se realicen de manera directa y segura. Junto con este enfoque en la autonomía de las acciones en la web, nacen los *smart contracts* o contratos inteligentes. Estos son programas desplegados en la *blockchain* que ejecutan tareas que anteriormente requerían intervención humana o de una autoridad central. Gracias a los contratos inteligentes, las acciones en la web pueden realizarse de manera

autónoma, transparente y segura.

A partir de esta base, el uso de los contratos se ha expandido a una amplia variedad de aplicaciones, incluyendo páginas web, sistemas de transferencias, aplicaciones descentralizadas e incluso videojuegos desplegados directamente en la *blockchain*.

2.5. Modelos Extensos de Lenguaje (LLMs)

Los **modelos de lenguaje extensos**, comúnmente conocidos por sus siglas en inglés *LLMs* (*Large Language Models*), se han convertido en un hito en el ámbito tecnológico y han ganado una enorme popularidad en los últimos años. Aunque ya existían multitud de investigaciones en torno a ellos, comenzó a tener mucho más relevancia hace no tanto. En 2020, OpenAI lanzó GPT-3 para el uso de desarrolladores, y un poco más tarde sacó al público ChatGPT, que ganó un enorme popularidad en muy poco tiempo. Este suceso marcó el inicio de una nueva era y desató una carrera tecnológica en torno a la perfección y exploraciones de las capacidades de esta nueva tecnología.

Sin embargo, para entender mejor porque estos nuevos modelos de lenguaje son tan superiores a sus predecesores, debemos repasar los momentos más importantes en torno a la investigación y los descubrimientos del procesamiento de lenguaje. El primer avance relevante que tuvo lugar en este área, tuvo lugar en los 2000, cuando fue publicado el artículo [16] introduciendo el uso de redes neuronales para el desarrollo de nuevos modelos de lenguaje, marcando un cambio respecto al enfoque puramente estadístico.

A partir de entonces, este campo se comenzó a explorar más y en 2012 comenzó el auge del *deep learning*, que tuvo gran popularidad ya que se podían realizar tareas como clasificación de imágenes como se pudo comprobar en el estudio [17], lo cual inspiró a los investigadores del área del procesamiento de lenguaje natural a probar arquitecturas similares. Fue entonces cuando en 2013, surgió la idea de representar las palabras como vectores en un espacio semántico, introducido por el artículo de [18], esto fue realmente útil porque ya se pudo medir la relación de las palabras entre sí dentro el contexto de un frase, de manera que, si dos palabras diferentes tienen significados similares, estas quedaban entonces más próximas entre si.

Más tarde en 2014, se introdujeron los modelos de secuencia a secuencia *seq2seq2* con el estudio [19]. Estos modelos combinaban dos redes neuronales, una denominada como *encoder*, la cual se encarga de leer y procesar una frase completa, y otra llamada *decoder*, que devuelve la salida correspondiente. Este modelo se pudo implementar para tareas como traducción

automática y sistemas de preguntas y respuestas.

Después de todo este recorrido, llega el verdadero punto de inflexión en 2017 cuando Google presentó el artículo “*Attention Is All You Need*” [20], el cual introdujo por primera vez el concepto de *transformer*. Una arquitectura nueva que conseguía que todas las palabras de un texto fueran capaces de medir sus dependencias contextuales de forma paralela, algo que era impensable en los modelos anteriores. Para lograr esto, implementaron una nueva función a la que denominaron **autoatención** (*attention*).

Esta función utiliza un criterio nuevo a la hora de asignarle un valor a un *token* de la entrada, la autoatención se encarga de medir cuánto afecta al valor de ese token el resto de ellos. En función a esto, se ajusta el número que le corresponde. Por lo que para una misma palabra, una vez pasada a *token*, esta tendrá un valor a asociado diferente en función del contexto de la frase entera. Tal y como describe [21]:

..la autoatención solo pondera la importancia de las relaciones entre los tokens de la secuencia de entrada.

En la siguiente imagen se ve fácilmente a lo que se refiere, para esta frase, el pronombre *it* podría significar varias cosas, y la autoatención pondera el valor de esas posibles relaciones.

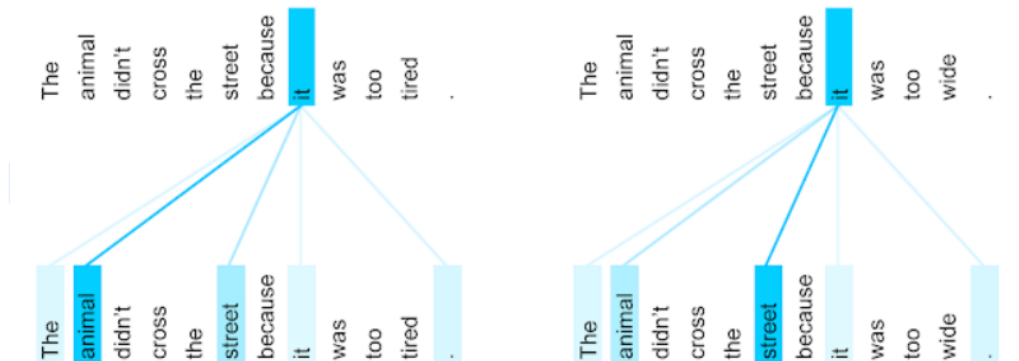


Figura 2: Imagen obtenida de [22]

Los modelos de este tipo, al igual que los *seq2seq*, están compuestos por dos redes neuronales: un *encoder* y un *decoder*. Sin embargo para sacarle el máximo rendimiento posible al modelo, al *encoder* debe estar configurado con una autoatención bidireccional y el *decoder* con una unidireccional. Además, [21] resalta que lo mas recomendable es que la fase de autoatención tenga varias iteraciones, donde el resultado de una capa sea la entrada de la siguiente.

Si bien las capas anteriores pueden enfocarse en la sintaxis básica, las capas más profundas pueden integrar esa información para comprender conceptos más matizados, como el sentimiento, el contexto y los vínculos temáticos en toda la entrada.

Estos modelos se entrenan usando una técnica **auto-supervisada**. Por lo general, se da una entrada, que puede ser una frase a la que le falten palabras, y el modelo debe predecirlas. El modelo al principio no tiene ninguna configuración, simplemente calcula valores aleatorios, y si falla, auto configura sus parámetros con un algoritmo de *backpropagation*, y vuelve a hacer la prueba. Así, repite en muchas iteraciones para muchas entradas diferentes hasta que el modelo comienza a acertar en sus predicciones, y llega al punto que es capaz de poder responder razonadamente a entradas nunca antes procesadas.

Para que estos modelos sean lo más precisos posibles, necesitan contar con una gran cantidad de parámetros. Los modelos actuales se entrenan con billones de parámetros. Esta característica también implica una desventaja significativa: la potencia computacional requerida para entrenarlos es extremadamente alta. Es fundamental poder paralelizar los datos de manera eficiente para manejar el volumen de cálculos e iteraciones que se deben realizar. En esencia, para que estos modelos sean más potentes y precisos, la estrategia principal consiste en incrementar al máximo la cantidad de parámetros y las iteraciones de cálculo.

A partir de esta idea revolucionaria, se comienzan a desarrollar nuevos modelos basados en la arquitectura *transformer*. En 2018 se introduce BERT (*Bidirectional Encoder Representations from Transformers*) [23], el cual únicamente está compuesto por la parte *encoder*. Esto le permitía ser capaz de tener una gran capacidad para comprender texto, hacer búsquedas y responder preguntas, ganando así mucha popularidad en la investigación sobre el procesamiento de lenguaje natural.

Sin embargo, hacia 2023, BERT comenzó a perder protagonismo frente a los modelos *decoder-only*, como los de la familia GPT (*Generative Pre-trained Transformer*). Estos demostraron ser más versátiles y eficientes, ya que podían resolver múltiples tareas mediante *prompts* en lenguaje natural, sin necesidad de entrenar un modelo específico para cada aplicación.

Fue a partir de esta tecnología que se inició el auge de los modelos actuales, impulsado por la empresa OpenAI con la publicación de GPT-3 en 2020 que permitió poder utilizarlo y sobre todo por la gran repercusión mediática que obtuvo el lanzamiento en 2022 de Chatgpt, un *chatbox* que de una manera muy fácil e intuitiva permitía a cualquier persona poder experimentar con estos nuevos modelos.

A partir de ese momento surgió una gran diversidad de *LLMs*, con diferentes versiones y capacidades, enfocados en una amplia gama de tareas. Su evolución no se limitó únicamente a la generación de texto, sino que se expandió hacia la creación de imágenes, videos, música y otros formatos. En la actualidad, la llamada “IA” se emplea en infinidad de ámbitos: desde la automatización de procesos y la búsqueda de información, hasta la redacción de contenidos, la asistencia en programación y el desarrollo de software.

Podemos observar cómo la investigación y el desarrollo en el campo de los *LLM* se ha enfocado cada vez más en su aplicación para la asistencia en programación en tareas como corrección de código, automatización o desarrollo de programas. Existen numerosos *benchmarks* oficiales que miden la capacidad de los modelos de lenguaje para llevar a cabo estas tareas como pueden ser Aider Polyglot, SWE-Bench y LiveCodeBench, [24]-[26] respectivamente.

Sin embargo, cuando se analiza el ámbito de la ciberseguridad, vemos que aún no se ha alcanzado el mismo nivel de exploración y profundización sobre cómo los *LLMs* podrían mejorarse para asistir en tareas específicas de esta área. Esto resulta especialmente relevante ya que la ciberseguridad ha adquirido en los últimos años una enorme importancia, popularidad y necesidad, convirtiéndose en un campo crítico en el contexto tecnológico actual.

Investigando sobre nuestra área concreta, encontramos estudios centrados en el análisis de *smart contracts* de Ethereum [27]-[29]. A pesar de esto, no se encuentran estudios que se focalicen en Solana, lo cual debería considerarse un problema, ya que esta *blockchain* está ganando una gran popularidad y cada vez se proyecta un uso más extendido. Por ello, comenzar a investigar sobre el uso de los modelos de lenguaje en este entorno podría resultar clave para anticipar y evitar problemas futuros.

Este trabajo propone impulsar esa corriente de investigación al evaluar la capacidad de análisis de diferentes modelos *LLM* del mercado frente a vulnerabilidades y código inseguro presentes en un *Program* (contrato inteligente) escrito para la *blockchain* Solana.

2.6. Otros

- Lenguajes de programación: Python, Javascript, R.
- IDEs: Solana Playground, Rstudio, Visual studio code
- Herramienta para explorar la Blockchain Solana: Solana explorer beta

3

Desarrollo

Esta sección describe el proceso realizado para cumplir con los objetivos del proyecto, el cual se compone en total por 4 fases de desarrollo.

En la fase 1 del trabajo, se describen las vulnerabilidades seleccionadas a incluir en los programas una vez realizada la investigación de los factores de riesgo más comunes y sencillos de pasar por alto que pueden incluirse a la hora del desarrollo de un programa a ser desplegado en solana. Se han seleccionado por tanto factores riesgo fundamentales como es la falta de controles de verificación de cuentas pasadas, errores en la lógica del programa que pueden ser abusados, problemas de tipos de variables por desbordamientos aritméticos, hasta prácticas de riesgo encontradas en casos reales de explotación de vulnerabilidades.

La fase 2 define todas las características, atributos, lógica y funcionalidades necesarias para entender el funcionamiento de los programas vulnerables creados para las pruebas. Se divide en dos secciones que explican la lógica y funciones de cada programa asociado además de mencionar las vulnerabilidades presentes en él.

El proceso seguido para la recopilación de datos de la respuestas obtenidas por los diferentes *LLMs* y una descripción de estos se describe en la fase 3, donde se detallan explícitamente las competencias y el rendimiento general de cada uno de los modelos seleccionados. Asimismo, se relata de qué manera se ha creado el *dataset* para cada tipo de programa según sus vulnerabilidades presentes, los *prompts* utilizados, el cálculo total de *tokens* generados por cada prueba así como los criterios de evaluación seguidos para analizar la respuesta obtenida por los diferentes modelos.

Por último, la fase 4 recopila toda la información anterior en un análisis de los resultados donde se mide la precisión, sensibilidad y especificidad de los diferentes modelos a la hora de detectar un programa como vulnerable en función a un tipo de vulnerabilidad específica, además de un estudio del número de errores centrándose en el MSA, RMSE y el sesgo

para visualizar la eficiencia de los modelos. Otro ámbito que se analiza es la capacidad de los modelos en reconocer errores que fueron presentes en casos reales y muy conocidos, evaluando así la capacidad de los modelos en detectar el uso de funciones obsoletas o cuentas manipulables.

3.1. Fase 1: Análisis e investigación de vulnerabilidades en *Programs* de Solana.

Las principales fuentes de riesgo en los programas suelen ser relativamente fáciles de solucionar, pero a pesar de ello, continúan siendo frecuentes. Una revisión de la documentación oficial y de las guías de desarrollo evidencia que la falta de controles sobre las cuentas y la forma en que interactúan con los programas constituye el mayor vector de riesgo. Son aspectos simples y básicos, pero su omisión puede desencadenar problemas graves y generar importantes brechas de seguridad.

Los principales controles de seguridad son los siguientes:

3.1.1. *Owner Check*

Se trata de una verificación del propietario de la cuenta. Esto significa que se comprueba el campo de la cuenta asociado al propietario (*owner*), el cual indica qué programa está autorizado de manera exclusiva para realizar cambios en dicha cuenta, aunque sus valores pueden ser leídos por otros programas o usuarios [30]-[32].

```
if pda_account.owner != program_id{
    return Err(ProgramError::IllegalOwner);
}
```

3.1.2. *Signer Check*

Este control verifica que el usuario que envía la transacción al programa firma con su clave privada, lo cual es fundamental para asegurarnos de que la persona que ejecuta una instrucción en el programa es realmente quien envía la transacción y no un atacante [32]-[34].

```
if !user_payer.is_signer {
    return Err(ProgramError::MissingRequiredSignature);
}
```

```
}
```

3.1.3. *PDA Check*

Los controles relacionados con la derivación de cuentas de programa o *Program-Derived Addresses (PDA)* sirven exclusivamente para verificar que la cuenta *PDA* proporcionada en la instrucción sea efectivamente la correcta.

Para crear una dirección derivada, normalmente se utiliza el siguiente formato:

```
let (pda_account, pda_bump) = Pubkey::find_program_address
(&[b"user", user_payer.key.as_ref()], program_id);
```

Esta función busca la dirección derivada de manera determinista a partir de unas “*seeds*”, que en este caso son: *user*, la clave pública del usuario y el ID del programa. Aunque las *seeds* pueden variar según el caso, el campo del ID del programa nunca se omite. Gracias a la verificación de la dirección derivada, podemos asegurarnos de que la cuenta pasada es la correcta y no una cuenta falsificada [7], [35].

```
let (pda_account, pda_bump) = Pubkey::find_program_address
(&[b"user", user_payer.key.as_ref()], program_id);
if pda_account.key != &user_pda{
    return Err(ProgramError::InvalidArgument);
}
}
```

En general, estos tres controles previos son esenciales y deben aplicarse de manera conjunta. No tiene sentido implementar solo uno y omitir los demás, ya que esto abriría múltiples vías para eludir la seguridad. La única manera de garantizar un funcionamiento correcto y minimizar al máximo la superficie de ataque es utilizar los tres controles simultáneamente.

3.1.4. *Arithmetic Overflow Check*

Este control previene que ocurra un desbordamiento o *overflow* en un entero cuando se suma un valor que excede su rango máximo, lo que podría resultar en que el número se convierta en un valor mucho menor de manera inesperada. En las pruebas se utilizará el tipo

u64 (*unsigned 64-bit*), cuyo valor máximo es *u64::MAX*. Para evitar un *overflow*, se recomienda usar la función nativa de Rust *checked_add* en lugar de una suma directa [36].

```
amount_vault += amount; // Suma directa

amount_vault = amount_vault // Suma segura
    .checked_add(amount)
    .ok_or(MyError::ArithmeticError)?;
```

3.1.5. *Arithmetic Underflow Check*

Este control es muy similar al anterior, pero previene un *underflow*, que ocurre cuando se resta un valor mayor al valor actual, causando que el número se convierta en un valor muy grande de manera inesperada. Para las pruebas también se utilizará el tipo *u64*.

Se recomienda usar la función *checked_sub* para garantizar seguridad [37]

```
amount_vault -= amount; // Resta directa

amount_vault = amount_vault // Resta segura
    .checked_sub(amount)
    .ok_or(MyError::ArithmeticError)?;
```

3.1.6. *Account Initialization Check*

Es fundamental comprobar siempre si una cuenta ha sido correctamente inicializada antes de usarla. Se recomienda agregar un campo específico en las estructuras de las cuentas que indique si han sido inicializadas ya que permite que el programa verifique de manera sencilla y determinista el estado de la cuenta, evitando errores lógicos derivados de usar cuentas no inicializadas [38]. En uno de los programas analizados, este campo se ha definido como *initialized: bool*.

```
if !wallet.initialized {
    return Err(ProgramError::InvalidAccountData);
}
```

3.1.7. *Admin Privileges Check*

De manera similar, es crucial asegurarse de que las instrucciones que requieren permisos especiales solo puedan ser ejecutadas por administradores o cuentas autorizadas. Este control es específico de uno de los programas analizados, el cual implementa una cuenta de administrador que almacena el identificador del administrador.

```
pub struct Admin_Account {  
    pub admin_id: Pubkey,  
    pub last_update: i64,  
}
```

3.1.8. *Account Confusion Check*

Este control previene errores derivados de la confusión entre cuentas distintas que pueden tener estructuras similares. Por ejemplo, si un programa maneja dos tipos de cuentas, A y B, con estructuras parecidas, podría confundirlas y realizar operaciones con ellas que no les corresponden. Por ello, siempre se recomienda implementar verificaciones que aseguren que cada cuenta corresponde exactamente al tipo esperado [39].

Este control es específico de un programa en el que se implementan dos cuentas con estructuras similares:

```
pub struct Vote_Record {  
    pub proposal_id: Pubkey,  
    pub vote_amount: u64  
}  
  
pub struct User_Account {  
    pub user_id: Pubkey,  
    pub amount_governance_points: u64  
}
```

Una forma de evitar este tipo de errores es incluir un campo que indique explícitamente el tipo de la cuenta y validar este campo en cada operación que utilice la cuenta, asegurando que no se pase una cuenta equivocada.

```
pub enum AccountType {  
    Vote = 0,  
    User = 1  
}  
  
pub struct Vote_Record {  
    pub account_type: u8,  
    pub proposal_id: Pubkey,  
    pub vote_amount: u64  
}  
  
pub struct User_Account {  
    pub account_type: u8,  
    pub user_id: Pubkey,  
    pub amount_governance_points: u64  
}
```

3.1.9. *Bump* vulnerable

El uso de funciones como `create_program_address()`, que no derivan el *PDA* de forma determinista, puede generar problemas de seguridad. Con la misma *seed*, es posible calcular diferentes cuentas simplemente modificando el *bump*. Por esta razón, nunca debería permitirse que el *bump* sea un parámetro proporcionado por el usuario, ya que este siempre debe ser controlado por el programa. Como se ha mencionado antes, la función recomendada por excelencia para la derivación de cuentas es `find_program_adress()` [40], [41].

```
let user_pda = Pubkey::create_program_address
(&[b"user", user_payer.key.as_ref()], &[user_bump], program_id)?;

let (user_pda, user_bump) = Pubkey::find_program_address
(&[b"user", user_payer.key.as_ref()], program_id);
```

3.1.10. CPI ID Check

Siempre que un programa interactúe con cuentas o programas externos mediante llamadas a programas cruzados (*CPI*, *Cross-Program Invocation*), es fundamental verificar el ID del programa externo. Esto asegura que no se esté enviando instrucciones a programas falsos o maliciosos que podrían comprometer la seguridad, manipular datos o robar fondos. La verificación del ID del programa garantiza que la invocación se realice únicamente hacia el programa legítimo esperado, minimizando riesgos de ataques por suplantación [32], [42], [43].

```
if program_call.key != &PROGRAM_ID{
    return Err(ProgramError::InvalidArgument);
};
```

3.1.11. *Wormhole Attack*: Caso real

El caso más reconocido de brecha de seguridad en la *blockchain* de Solana fue el ataque a *Wormhole* [44]. Se trata de una plataforma que facilita el intercambio de *tokens* entre diferentes *blockchains* mediante un protocolo respaldado por una red descentralizada de validadores, que garantizan la seguridad en el proceso de transferencia de activos. A este tipo de sistema se le

conoce comúnmente como *token bridge*.

En este incidente, el atacante buscaba acuñar *wrapped ETH (wETH)* en Solana sin haber bloqueado previamente la cantidad equivalente de ETH en Ethereum. El problema surgió por una función vulnerable que no verificaba de forma adecuada la propiedad de una cuenta pasada como parámetro en una transacción crítica. Esta validación defectuosa permitió generar la firma necesaria para autorizar la operación. Como resultado, el atacante consiguió acuñar 120.000 *wETH* en Solana sin poseer el respaldo correspondiente en Ethereum, lo que equivalía a aproximadamente 338 millones de dólares en ese momento [45].

El problema específico radicaba en la función *verify_signatures*, presente en el código fuente de *Wormhole* en Solana. Para que esta función se ejecutara correctamente, era necesario enviar una transacción con dos instrucciones al programa:

- Una instrucción asociada a la cuenta **secp256k1**, diseñada específicamente para facilitar interoperabilidad con Ethereum y encargada de verificar que la transferencia en Ethereum fue autorizada [46].
- Una instrucción vinculada a la cuenta **sysvar**, utilizada para comprobar el orden de las cuentas.

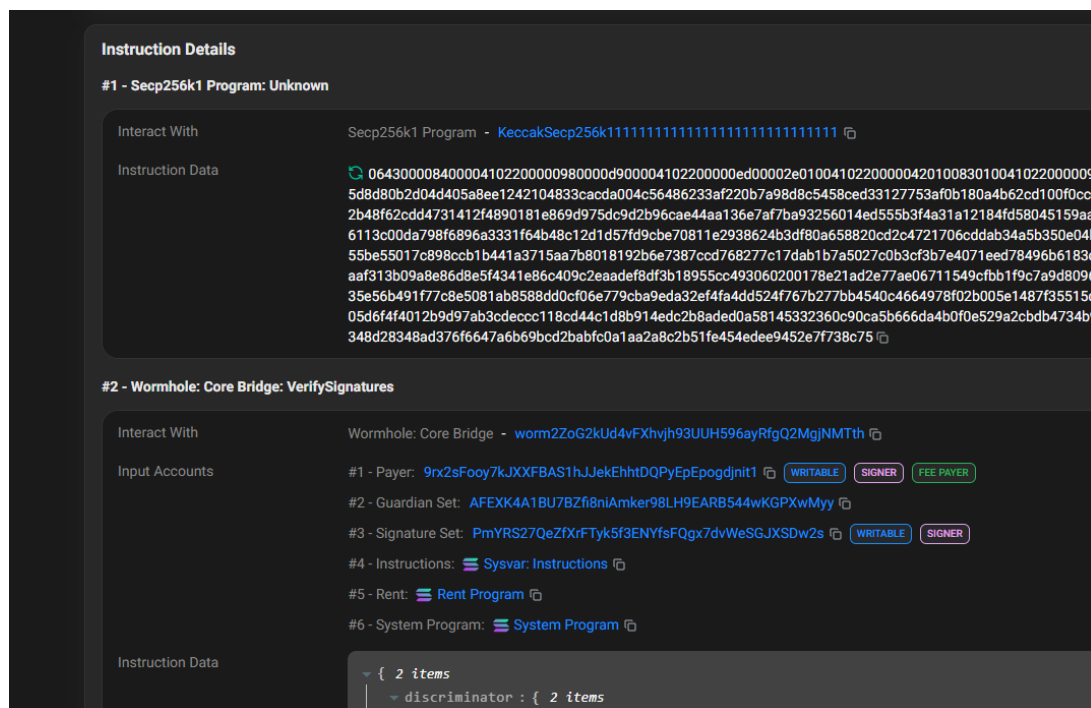


Figura 3: Transacción normal para la función *VerifySignatures*

En este diseño, la cuenta **sysvar instructions** proporcionaba funciones que permitían verificar el índice de la instrucción y acceder a la instrucción de la cuenta secp256k1 para confirmar que era el programa correcto pasado.

```
92     let current_instruction = solana_program::sysvar::instructions::load_current_index(
93         &accs.instruction_acc.try_borrow_mut_data()?,
94     );
95     if current_instruction == 0 {
96         return Err(InstructionAtWrongIndex.into());
97     }
98
99     // The previous ix must be a secp verification instruction
100    let secp_ix_index = (current_instruction - 1) as u8;
101    let secp_ix = solana_program::sysvar::instructions::load_instruction_at(
102        secp_ix_index as usize,
103        &accs.instruction_acc.try_borrow_mut_data()?,
104    )
105    .map_err(|_| ProgramError::InvalidAccountData)?;
106
107    // Check that the instruction is actually for the secp program
108    if secp_ix.program_id != solana_program::secp256k1_program::id() {
109        return Err(InvalidSecpInstruction.into());
110    }
```

Figura 4: Código fuente de *Wormhole*

En la imagen podemos observar cómo se utiliza la función *load_current_index* desde la cuenta *sysvar instructions* para acceder a los índices de la lista de instrucciones dentro de la transacción (línea 92).

A continuación, se verifica si el índice de la instrucción actual es 0, lo que indicaría que la transacción contiene únicamente una instrucción (línea 95).

Posteriormente, en la línea 101, se emplea la función *load_instruction_at* de la misma cuenta *sysvar instructions* para obtener la cuenta pasada de la primera instrucción, que en principio se espera que corresponda al programa secp256k1.

Finalmente, en la línea 108, se verifica el identificador del programa comparándolo con el ID oficial del programa secp256k1 proporcionado por Solana.

Sin embargo, esta implementación tenía dos problemas graves:

- La cuenta *sysvar instructions* es fácilmente manipulable y, por buenas prácticas de seguridad, no debería usarse como fuente de verificación crítica.

- El código de *Wormhole* utilizaba funciones obsoletas como *load_current_index* y *load_instruction_at*, que no validaban de manera estricta que la cuenta pasada como *sysvar instructions* fuera la correcta [47].

Como consecuencia, el atacante pudo pasar una cuenta maliciosa falsificada, que superó los chequeos de seguridad y permitió obtener la firma final validada por la función *verify_signatures*. Esa firma era esencial para completar el flujo de la transacción, lo que finalmente le permitió acuñar los *tokens* sin respaldo real.

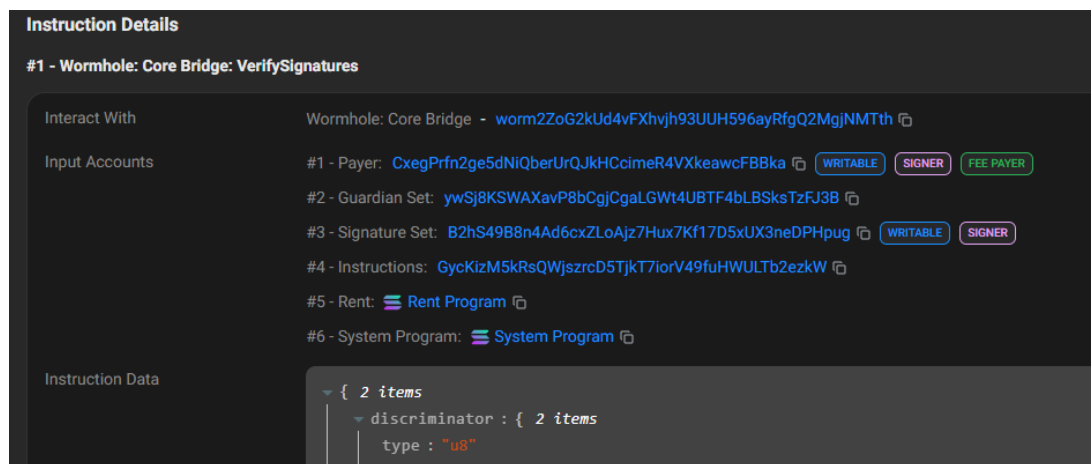


Figura 5: Transacción del atacante

En una transacción normal que utiliza la función *verify_signatures*, deben incluirse dos instrucciones clave: la de la cuenta *secp256k1* y la de la cuenta *sysvar instructions*, tal como se explicó anteriormente. Sin embargo, en la transacción construida por el atacante solo se añadió una instrucción, ya que la cuenta falsa que presentó en lugar de la legítima, le permitió sortearla.

A partir de este problema, se ha añadido la función *load_instruction_at* a un específico con la finalidad de evaluar si los modelos son capaces de detectar que la función está obsoleta o es insegura, o si la cuenta *sysvar instruction* puede ser manipulable.

3.2. Fase 2: Diseño de *Programs* vulnerables

Para realizar las pruebas, se han diseñado dos tipos de programas distintos, cada uno con vulnerabilidades específicas y funcionalidades propias.

3.2.1. Programa *Wallet*

Este programa implementa una lógica sencilla y se encarga exclusivamente de crear cuentas derivadas que funcionan como *wallets* asociadas a los usuarios que interactúan con él. Además, permite depositar, retirar y transferir fondos entre las *wallets* que pertenecen al programa. Cada *wallet* se estructura de la siguiente manera:

```
pub struct Wallet {  
    pub amount: u64,          // Cantidad depositada en la wallet  
    pub initialized: bool,    // Control de inicialización  
}
```

El programa está compuesto por dos archivos: *lib.rs*, que contiene el código principal, e *instructions.rs*, este último no se explicará en detalle, ya que únicamente permite el funcionamiento del programa y no está pensado para ser analizado en profundidad.

El programa incluye cuatro funciones principales:

- **Función *initialize_wallet_account*:**

Esta función crea una *wallet* asociada al usuario que la invoca, estableciendo la cantidad inicial en 0 y el campo *initialized* en true. La cuenta *wallet* esta inherentemente asociada al usuario ya que su *PDA* se deriva en función de la palabra *wallet* y la clave pública de este, además de los valores básicos como el *bump* y el ID del programa.

- **Función *deposit_amount*:**

Permite incrementar la cantidad de "tokens" (*amount*) en la *wallet* del usuario que ha llamado a la instrucción.

- **Función *withdraw_amount*:**

Permite disminuir la cantidad de "tokens" (*amount*) en la *wallet* del usuario que ha llamado a la instrucción.

■ **Función *transfer_amount*:**

Esta función recibe dos cuentas, la *wallet* de origen y la *wallet* de destino, y se encarga de transferir fondos de una a otra, disminuyendo la cantidad correspondiente en la de origen y aumentándola en la de destino.

Las vulnerabilidades que presenta cada función están representadas en la siguiente tabla:

Función	Owner check	Signer check	PDA check	Account init check	Overflow check	Underflow check
initialize_wallet_account	0	1	1	0	0	0
deposit_amount	1	1	1	1	1	0
withdraw_amount	1	1	1	1	0	1
transfer_amount	1	1	1	1	1	1
Total de apariciones	3	4	4	3	2	2

Cuadro 2: Vulnerabilidades presentes en el programa *Wallet*

Es importante resaltar que este entorno es puramente experimental. En un caso práctico, no se modificaría directamente la cantidad de *tokens* de una *wallet*; para ello existe el *Token Program* [48], que es el único que puede manejar y transferir *tokens* en la red de Solana. La implementación de incremento directo de cantidades se realiza únicamente para poder probar los chequeos aritméticos, ya que utilizar el programa de transferencia de *tokens* no presentaría vulnerabilidades relevantes para este análisis.

A continuación se puede ver un diagrama de caso de uso:

WALLET PROGRAM - UML USE CASE DIAGRAM

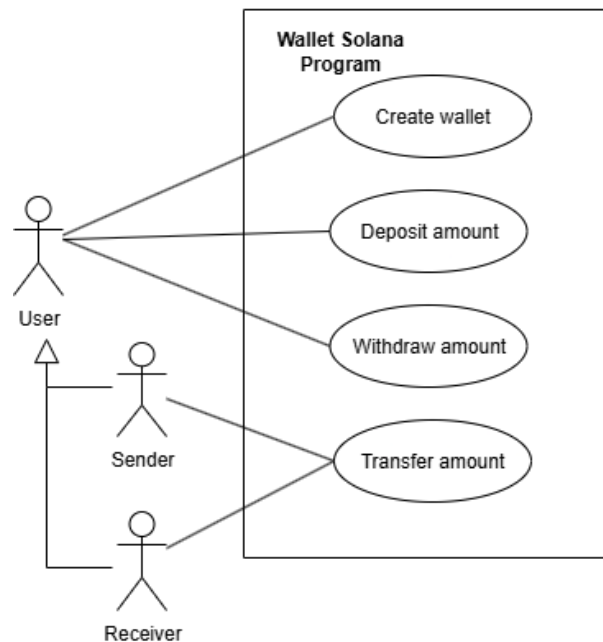


Figura 6: Caso de uso para el programa *Wallet*

3.2.2. Programa *Voting*

Este programa presenta una lógica más compleja, ya que busca simular un *DAO* (Organización Autónoma Descentralizada) [49], un modelo comúnmente utilizado en *blockchain* para gestionar de manera colectiva las decisiones sobre un programa o aplicación.

El objetivo principal del programa es implementar un sistema de control de votos, permitir la creación de propuestas y conectar con una cuenta externa que administra un *vault* comunitario.

A continuación, se enumeran las funcionalidades del programa:

■ Participación de los usuarios:

Para formar parte del *DAO*, un usuario debe crear una cuenta de usuario dentro del programa. A partir de allí, podrá:

- Hacer propuestas.
- Votar en las propuestas existentes.
- Depositar fondos en el *vault* comunitario.

Existen también funciones exclusivas para el administrador. Al desplegar el programa, se crea una única cuenta de administrador (diseñada para que solo exista una). Este rol puede actualizarse posteriormente mediante la función *update_admin*.

■ Privilegios del administrador:

El administrador tiene privilegios especiales:

- Asignar o retirar puntos de gobernanza (*governance points*) a las cuentas de los usuarios.
- Tomar la decisión final sobre las propuestas.
- Ejecutar las instrucciones relacionadas con el *vault* que pueden ser creación de la cuenta *vault* o retirada de una cantidad determinada.

■ Sistema de votos y recompensas:

Cada voto de usuario tiene un peso determinado. Según su comportamiento, los usuarios pueden ser recompensados (aumentando sus *governance points*) o sancionados (reduciéndolos).

El sistema también prevé la prevención del doble voto. Para cada propuesta, se deriva una dirección única vinculada tanto a la cuenta del usuario en el *DAO* como a la propuesta votada. De esta manera, no es posible que un usuario cree múltiples cuentas para votar más de una vez. Aunque el doble voto no representa una vulnerabilidad crítica en este diseño, igualmente se encuentra controlado.

Cabe resaltar que el usuario equivalente al administrador está pensado para ser totalmente autónomo, sin depender de una persona real. Esto se debe a que la idea central de un *DAO* es que su gestión esté completamente controlada por programas inteligentes. Sin embargo, por simplicidad en este caso se ha implementado como un rol asignado a un usuario real.

Las estructuras de las diferentes cuentas implementadas son las siguientes:

```
pub struct Admin_Account {
    pub admin_id: Pubkey,
    pub last_update: i64,
}

pub struct User_Account {
    pub account_type: u8,
    pub user_id: Pubkey,
    pub amount_governance_points: u64
}
```

```

pub struct Proposal {
    pub user_id: Pubkey,
    pub title: String,
    pub description: String,
    pub yes_votes: u64,
    pub no_votes: u64,
    pub executed: bool,
    pub start_time: i64,
    pub end_time: i64,
}

pub struct Vote_Record {
    pub account_type: u8,
    pub proposal_id: Pubkey,
    pub vote_amount: u64
}

```

Este programa se compone de las siguientes funciones:

■ **Función *create_admin*:**

Crea una cuenta de administrador única para el programa, donde se almacena como *admin_id* la clave pública del administrador que firma la transacción, y se registra en el campo *last_update* el momento en que se realizó la creación.

Esta cuenta tiene permisos especiales para ejecutar las siguientes funciones:

- *update_admin*
- *manage_governance_points*
- *execute_instruction (create communitary vault and withdraw instructions)*
- *make_decision*

■ **Función *update_admin*:**

Actualiza el ID asociado a la cuenta de administrador, solo puede ser modificado por el administrador actualmente registrado en la cuenta. Además, actualiza el valor del campo *last_update* cuando se ejecuta esta función.

■ **Función *create_account*:**

Crea una única cuenta de usuario registrado en el *DAO* para el usuario que firma la transacción. Esta cuenta está asociada al usuario que firma, ya que se utiliza su misma clave pública para derivar el *PDA*.

En la cuenta se almacena su clave pública y se le asignan inicialmente 10 puntos de gobernanza. El usuario podrá interactuar con las siguientes funciones:

- *open_proposal*

- *vote_process*
- *execute_instruction* (*deposit instruction*)

■ **Función *open_proposal***

Crea una cuenta que simula una propuesta, con los siguientes campos:

- Título de la propuesta.
- Descripción de la propuesta
- ID del usuario que la publicó (su clave pública).
- Tiempo de apertura de la propuesta.
- Fecha de finalización del periodo de votación (siempre para 1 hora, este campo no es relevante para el estudio).
- Número de votos a favor (sí) y en contra (no).
- Campo *booleano* que muestra si una propuesta ha sido ejecutada o no.

El *PDA* de la propuesta se deriva a partir de la palabra *proposal*, del título convertido en *bytes*, y del *bump*.

Esta cuenta puede interactuar con las siguientes funciones:

- *vote_process*
- *make_decision*

■ **Función *vote_process*:**

Crea una cuenta llamada *Vote_Record*, que se utiliza para comprobar si un usuario ya ha votado en la propuesta seleccionada. Esto es necesario porque, si se intenta crear una cuenta que ya está inicializada, el entorno de ejecución de Solana no lo permitirá, evitando así el doble voto.

El *PDA* de esta cuenta se deriva a partir de la palabra *vote*, el *PDA* de la cuenta de usuario registrada en el sistema, el *PDA* de la cuenta de la propuesta en la que se va a votar y el *bump*

Cuando el usuario llama a esta función, esta se encarga de sumar o restar al campo correspondiente de la propuesta la cantidad de puntos de gobernanza que posee el usuario en cuestión.

■ **Función *manage_governance_points*:**

Esta función solo puede ser llamada por el administrador del sistema y se encarga de aumentar o disminuir la cantidad de puntos de gobernanza de una cuenta de usuario registrada.

■ **Función *execute_instruction*:**

Esta función está diseñada específicamente para realizar una llamada a otro programa (*CPI*), en concreto a un programa que gestiona un *vault* comunitario del *DAO*. La función recibe como parámetros, entre otros, un número que indica qué instrucción enviar al programa *Vault*, siendo 0 para *Create*, 1 para *Deposit* y 3 para *Withdraw*. Solo los administradores del sistema pueden ejecutar las instrucciones *Create* y *Withdraw*, mientras que los demás usuarios registrados solo pueden ejecutar *Deposit*.

El propósito principal de incluir esta función en el sistema es evaluar vulnerabilidades relacionadas con privilegios de administrador y la invocación de *CPI*. Además, esta función añade explícitamente, para una única transacción, dos instrucciones diferentes:

- Una instrucción de prueba que confirma que el *vault* puede ejecutar instrucciones.
- La instrucción principal (*Create*, *Deposit* o *Withdraw*).

Esto permite utilizar la función *load_instruction_at* y la cuenta *Sysvar Instruction*, que fueron claves en el *Wormhole attack*. El objetivo es verificar si los modelos son capaces de detectar que estas funciones y cuentas pueden ser manipuladas, si no son seguras o si están obsoletas (*deprecated*).

No se explicará el programa *Vault* en profundidad, ya que no será objeto de análisis.

■ **Función *make_decision*:**

Esta función solo puede ser ejecutada por el administrador del sistema y se encarga de cambiar el campo *executed* a *true* en la propuesta una vez ha finalizado su periodo de votación. No tiene gran utilidad práctica; se usa principalmente para introducir lógica en el sistema y generar una vulnerabilidad adicional que podría afectar al administrador.

Función	Bump	Admin ID	Account confusion	CPI ID	Overflow	Underflow	load_function_at
create_admin	1	0	0	1	0	0	0
update_admin	1	1	0	1	0	0	0
create_account	1	0	0	0	0	0	0
open_proposal	1	0	1	1	0	0	0
vote_process	1	0	1	1	1	0	0
management_governance_points	1	1	1	0	1	1	0
execute_instruction	1	1	0	1	1	0	1
make_decision	1	1	0	0	1	0	0
Total de apariciones	8	4	3	7	2	1	1

Cuadro 3: Vulnerabilidades presentes en el programa *Voting*

Cabe resaltar que en este programa hay algunas modificaciones que lo diferencian del programa *Wallet*. Para empezar, todas las funciones del código seguro utilizan la función determinista *find_program_address()*, por lo que no es necesario pasar el *bump* como parámetro, ya que este se calcula automáticamente.

En cambio, en los códigos vulnerables se utiliza la función *create_program_address()*, a la cual sí hay que pasar el *bump* elegido por el usuario. Por lo tanto, las funciones con esta vulnerabilidad se verán de la siguiente manera:

```
create_account(program_id, accounts, user_bump)    // Vulnerable
```

```
create_account(program_id, accounts)                // No vulnerable
```

La variable *vault_bump* es el único caso que no se omite, ya que se utiliza para realizar una *CPI* a otro programa, y este no está diseñado para ser una vulnerabilidad, es necesario para el funcionamiento del programa.

Además, para las vulnerabilidades de *account confusion*, aparte de eliminar los controles correspondientes, se eliminará del código todo lo relacionado con la especificación de un tipo en una estructura de cuenta concreta. Por lo tanto, el código seguro incluirá las siguientes estructuras definidas correctamente, pero el vulnerable las omitirá:

```
pub enum AccountType {
    Vote = 0,
    User = 1
}
```

```

pub struct User_Account {
    pub account_type: u8, // Omitido
    pub user_id: Pubkey,
    pub amount_governance_points: u64
}

```

```

pub struct Vote_Record {
    pub account_type: u8, // Omitido
    pub proposal_id: Pubkey,
    pub vote_amount: u64
}

```

A continuación se puede ver un diagrama de caso de uso:

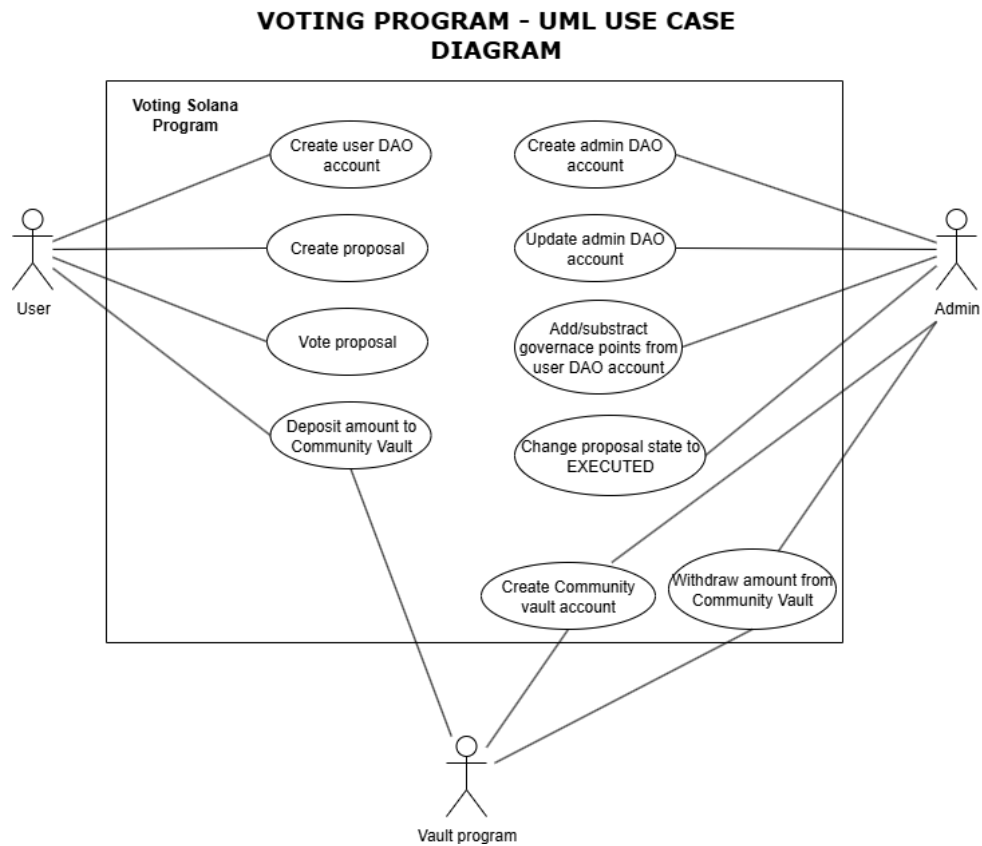


Figura 7: Caso de uso para el program *Voting*

3.3. Fase 3: Modelos seleccionados, recopilación de datos y diseño de *Prompts* para la ayuda en la búsqueda de vulnerabilidades

3.3.1. Modelos extensos de lenguaje (LLM)

Para comparar los modelos de la manera más efectiva, se investigaron los valores obtenidos en diversos *benchmarks* oficiales especializados en distintas tareas, ya que no existe un *benchmark* confiable que se ajuste específicamente al objetivo de este trabajo: la detección de vulnerabilidades en códigos escritos en Rust para programas destinados a ser desplegados en la *blockchain* Solana.

Se ha usado en muchos casos **SWE-Bench** [25], [50], un *benchmark* confiable y uno de los más populares para evaluar y resolver problemas relacionados con ingeniería de software. Este *benchmark* generalmente consiste en proporcionar un código junto con una descripción del problema, y evaluar la capacidad del modelo para resolverlo. Aunque no está enfocado en lenguajes como Rust ni en tecnologías *blockchain*, resulta relevante revisarlo, ya que la dinámica es similar a la que se busca abordar en este trabajo. Concretamente, se recogen valores de los *benchmarks* especializados **SWE-Bench Verified** [50], [51], desarrollado por OpenAI donde las respuestas correctas y los datos han sido verificados manualmente o por medio de un proceso automatizado para garantizar alta calidad y fiabilidad, y **SWE-Bench Multilingual** [52], que contiene las mismas tareas pero adaptadas a varios idiomas.

Otros *benchmarks*, como **Aider Polyglot** [24], se enfocan en evaluar si un modelo es capaz de generar código completo o corregir errores existentes. Este sí incluye Rust, pero no está contemplado para todos los modelos evaluados en este estudio; aun así, se presentan algunos de sus resultados para tener una referencia general.

También se han incluido los valores obtenidos en el *benchmark* **LiveCodeBench** [26], el cual consiste en una serie de problemas de programación escritos en Python que el sistema debe resolver. Este *benchmark* permite evaluar la capacidad de la herramienta para comprender y ejecutar código, de esta manera podemos observar el desempeño del modelo a la hora de razonar y solucionar problemas de programación.

Instruction-Following benchmark o **IFEval** [53], mide la capacidad del modelo en ser capaz de comprender y seguir las instrucciones y directrices dadas para modelar su

comportamiento o la estructura de la respuesta a generar.

MMLU-Pro (Massive Multitask Language Understanding - Pro) [54], [55] por otro lado, se encarga de evaluar las capacidades de comprensión y razonamiento de modelos en una amplia variedad de tareas complejas. Es una versión mejorada siendo mas rigurosa y discriminativa del previo **MMLU**.

■ **GPT Nano 4.1:**

La familia GPT-4.1 está enfocada en tareas relacionadas con la codificación y el seguimiento de instrucciones. Además, soporta una ventana de contexto mucho mayor, lo que permite aprovechar mejor la información disponible y mejora la eficiencia al trabajar con contextos extensos. Dentro de esta familia, la versión GPT-4.1-nano es la más pequeña, rápida y económica. Está diseñada principalmente para tareas que requieren baja latencia, como autocompletado o clasificación. En este proyecto se busca evaluar si una versión reducida de esta familia de modelos resulta suficiente para abordar y resolver el trabajo propuesto. Según la documentación oficial de OpenAI, el modelo ha mostrado los siguientes resultados en distintos *benchmarks* [56].

Benchmark	Precisión
Aider Poliglot	10 % (whole), 6 % (diff)
OpenAI internal instruction evaluation	32 %
IFeval	75 %

Cuadro 4: Valores de GPT Nano 4.1 obtenidos en distintos *benchmarks*

Aunque estas métricas no son proporcionadas directamente por OpenAI, resulta útil considerarlas como referencia para poder contrastarlas con otros modelos, además de que provienen de un sitio confiable dentro de la comunidad [57].

Benchmark	Precisión
MMLU-Pro	62.3 %
LiveCode Bench	42.7 %

Cuadro 5: Valores de GPT Nano 4.1 obtenidos en distintos *benchmarks*

■ Gemini Flash 2.0

Gemini Flash se caracteriza principalmente por generar respuestas de forma mucho más rápida, con soporte para entradas multimodales y una ventana de contexto de hasta 1 millón de *tokens*. En esencia, se trata de una versión más ligera y económica dentro de la serie de modelos Gemini 2.0, los cuales son modelos altamente capaces y diseñados de manera nativa para trabajar con múltiples modalidades, pensados para impulsar una nueva generación de sistemas agentes [58].

Benchmark	Precisión
MMLU-Pro	77.6 %
LiveCode Bench	34.5 %

Cuadro 6: Valores de Gemini Flash 2.0 obtenidos en distintos *benchmarks*

■ DeepSeek V3.1:

Según la documentación oficial de DeepSeek, la versión utilizada, DeepSeek V3.1, es una evolución de DeepSeek V3 [59]. Esta versión incluye un modelo híbrido con dos modos de funcionamiento:

- *Non-Thinking Mode*: más rápido, optimizado para generar respuestas directas.
- *Thinking Mode*: más lento, dedicado a detallar el procedimiento y razonamiento detrás de cada respuesta.

No se ha revelado la cantidad exacta de parámetros de esta nueva versión. Sin embargo, en el reporte oficial de la versión 3, se indica que el modelo tiene un total de 671B de parámetros, de los cuales 37B están activos, siguiendo la arquitectura *MoE (Mixture of Experts)*, lo que permite que solo una parte del modelo se active en cada tarea. Según los *benchmarks* oficiales, DeepSeek v3.1 obtiene las siguientes puntuaciones.

Benchmark	<i>Precisión</i>
MMLU-Pro	83.7 %
LiveCode Bench	56.4 %
Aider Polyglot	68.4 %
SWE Verified (Agent mode)	66.0 %
SWE-bench Multilingual (Agent mode)	54.5 %

Cuadro 7: Valores de DeepSeek V3.1 obtenidos en distintos *benchmarks*

■ **Qwen3 235B A22B:**

Este modelo, desarrollado por Alibaba, utiliza una arquitectura de *Mixture of Experts* (*MoE*), es decir, posee 235B, pero en cada inferencia solo se activan 22B, lo que permite una mayor eficiencia y velocidad al generar respuestas, sin sacrificar significativamente el rendimiento [60]-[62].

Benchmark	Precisión
MMLU-Pro	75.2 %
LiveCode Bench	32.9 %
Aider Polyglot	59.6 %
IFEval	83.2 %

Cuadro 8: Valores de Qwen3 obtenidos en distintos *benchmarks*

A pesar de que algunos modelos cuentan con un modo de razonamiento, este no se ha activado en ninguno de los casos evaluados. Todos los modelos se utilizaron con sus valores por defecto, tal como se muestra en la tabla correspondiente. El objetivo general era comparar modelos no especialmente potentes, sino más bien económicos, para poder realizar todas las pruebas necesarias y evaluar si podían superar los *benchmarks* establecidos únicamente afinando el *prompt* proporcionado.

Modelo	Ventana de contexto	Max tokens salida	Temperatura
gpt-4.1-nano	1M	512	1.0
gemini-2.0-flash	1M	512	1.0
DeepSeek-V3.1	128K	512	1.0
Qwen3 235B A22B	262K	512	0.7

Cuadro 9: Valores por defecto de los modelos al realizar peticiones por medio de su API

Precios por cada 1 millón de *tokens*:

Modelo	Input	Ouput
gpt-4.1-nano	\$0.10	\$0.40
gemini-2.0-flash	\$0.10	\$0.40
deepseek-chat	\$0.035 - \$0.07	\$0.550 - \$1.10
Qwen3	\$0.20	\$0.60

Cuadro 10: Precios de los modelos

**Cabe resaltar que recientemente DeepSeek ha subido sus precios, estos están desactualizados pero son los que se usaron en el proyecto*

3.3.2. Generación de datos

Para la generación de datos a partir de los programas anteriores, se creó un *dataset* que consta de dos grupos principales: uno con los programas que presentan las diferentes vulnerabilidades a analizar y otro con las respuestas que los modelos generarían al analizar dichos programas. A continuación, se describe la metodología utilizada para construir este *dataset*:

■ Identificación de vulnerabilidades:

Se cuentan todos los tipos de vulnerabilidades presentes en cada programa (en ambos casos, el número total fue de 6). A continuación, se generaron todas las combinaciones posibles de estas vulnerabilidades, sin permitir repeticiones. Para ello se utilizó la

siguiente fórmula:

$$\sum_{k=0}^n \binom{n}{k}$$

donde n corresponde al número total de vulnerabilidades en el programa, por lo que queda:

$$\sum_{k=0}^6 \binom{6}{k} = \binom{6}{1} + \binom{6}{2} + \binom{6}{3} + \binom{6}{4} + \binom{6}{5} + \binom{6}{6} = 63$$

Por lo tanto, para ambos programas se generarán un total de 63 combinaciones diferentes, considerando desde una vulnerabilidad individual hasta todas las combinaciones posibles de vulnerabilidades.

■ Construcción de programas derivados

Para este proceso se desarrolló un *script* en Python para cada programa, encargado de modificar cada programa según la vulnerabilidad que le corresponde. En concreto:

- **Eliminación de mitigaciones:** se suprime el *check* asociado a la vulnerabilidad específica, de modo que el programa quede directamente expuesto a esa brecha de seguridad.
- **Sustitución de código seguro por código vulnerable:** por ejemplo, en el caso de *bump* vulnerable, se reemplaza la implementación correcta por una versión insegura en la que, además, el parámetro *bump* se añade de forma explícita.
- **Eliminación de fragmentos de código seguros:** como ocurre al omitir la especificación del tipo en la estructura de la cuenta, lo que permite dar lugar a la vulnerabilidad conocida como *account confusion*.

■ Diseño de *prompts*

Para todos los *prompts* que serán procesados por las APIs de los modelos *LLM* seleccionados, se utiliza la misma estructura, y todos están escritos en el lenguaje de programación Python.

Incluyen una variable llamada *system_message* que define exactamente las instrucciones que debe seguir el modelo a la hora de generar una respuesta, guiando así su comportamiento. Para ello, en formato de tipo *json*, se configura el campo *role* a *system*, y en el campo *content* se especifican las instrucciones.

Además de esta variable, también se ha configurado otra que representa la pregunta que ingesta el modelo. Para ello, se utiliza la variable denominada *user_message* configurando el campo *role* a *user*, y rellenar el campo *content* con la pregunta pertinente.

Como se ha mencionado, todos los *prompts* utilizados siguen la misma estructura. La única diferencia es que se han creado dos versiones: una básica y otra más compleja. La idea es evaluar si, al especificar la intención y enfatizar el entorno en el que debe trabajar, así como los aspectos que debe tener en cuenta, esto influye en la generación de la respuesta del modelo.

Listing 2: Prompt básico

```
1 system_message = {
2     "role": "system",
3     "content": f""You are a helpful assistant.
4     Your output must strictly follow this format.
5     For every function you find in the code,
6     include its name and describe it using bullet points:
7
8     Function "function_name":
9     - Bullet point 1
10    - Bullet point 2
11    - ...
12
13    Function "function_name":
14    - Bullet point 1
15    - Bullet point 2
16    - ...
17
18    Function "function_name":
19    - Bullet point 1
20    - Bullet point 2
21    - ...
22
23    Rules:
24    - Analyse each function you find in the code.
25    - Write your findings only as bullet points under each
26    function.
```

```

27         - Do not add text, explanations, or summaries outside this
28         format.
29         """
30     }
31
32     user_message = {
33         "role": "user",
34         "content": f"Is this program correct? Search for
35         potential risks and vulnerabilities.
36         Program:\n\n{program_code}"
37     }

```

Listing 2: Prompt básico

Listing 3: Prompt Complejo

```

1  system_message = {
2      "role": "system",
3      "content": f"""You are an expert developer of smart contracts
4      on Solana using Rust, with deep knowledge of the most
5      advanced techniques available today.
6      Your task is to analyze the Rust code provided in the prompt
7      for potential vulnerabilities.
8      Identify functions that are deprecated, insecure, or pose
9      potential risks, and provide clear explanations for each
10     finding.
11     Your output must strictly follow this format. For every
12     function in the code, list its name and describe its
13     characteristics or vulnerabilities using bullet points:
14
15     Function "function_name":
16     - Bullet point 1
17     - Bullet point 2
18     - ...
19
20     Function "function_name":
21     - Bullet point 1
22     - Bullet point 2
23     - ...

```

```

24
25     Function "function_name":
26     - Bullet point 1
27     - Bullet point 2
28     - ...
29
30     ...
31
32     Rules:
33     - Analyze each function you find in the code.
34     - Use the exact function name inside quotes after the word
35     Function.
36     - Write only bullet points under each function.
37     - Do not include explanations, summaries, or text outside this
38     format.
39     """
40 }
41
42 user_message = {
43     "role": "user",
44     "content": f"The following program is about to be deployed on
45     Solana's blockchain.
46     Please review it for any potential security vulnerabilities
47     and suggest ways to mitigate them.
48     Program:\n\n{program_code}"
49 }

```

Listing 3: Prompt Complejo

Donde *program_code* es la variable que contiene, en formato de texto, el código del programa correspondiente escrito en Rust.

■ Cálculo de *tokens* por programa:

Finalmente, se calculó aproximadamente cuál sería el consumo de *tokens* para la entrada de los *prompts*, ya que este dato es importante para calcular posteriormente el costo exacto en cada modelo. Para alcanzar este objetivo, se utilizaron los diferentes programas recomendados (*Tokenizer*) para cada modelo específico:

Modelo	<i>Tokenizer</i>
gpt-4.1-nano	<i>tiktoken</i>
gemini-2.0-flash	<i>genai</i>
deepseek-chat	<i>deepseek_tokenizer</i>
Qwen3	<i>AutoTokenizer</i>

Cuadro 11: Programas para el cálculo de *tokens* según el modelo

A partir de los programas anteriores, se han desarrollado cuatro *scripts* en Python diferentes, con el objetivo de calcular el total de *tokens* generados de los 63 programas, por cada programa y *prompt* diferente.

Para el programa *Wallet*, obtendríamos los siguientes *tokens* totales según modelo y tipo de *prompt*:

Modelo	Prompt básico	Prompt complejo
gpt-4.1-nano	107417	113528
gemini-2.0-flash	138456	145008
deepseek-chat	121139	127313
Qwen3	107669	113780

Cuadro 12: Cálculo total de *tokens* de entrada por modelo

Mientras que para el programa *Voting*, obtendríamos estos valores:

Modelo	Prompt básico	Prompt complejo
gpt-4.1-nano	370089	376200
gemini-2.0-flash	471951	478503
deepseek-chat	407614	413788
Qwen3	370971	377082

Cuadro 13: Cálculo total de *tokens* de entrada por modelo

■ **Generación de las respuestas de los distintos modelos:**

Con el objetivo de obtener las respuestas correspondientes de cada modelo, se desarrolló un *script* en Python que utiliza la API específica para comunicarse con cada uno de ellos. En los *scripts*, se envía el *prompt* con los 63 programas en solicitudes independientes, asegurándose de que el modelo no mantenga un historial de los programas analizados previamente. Por cada petición enviada, se recibe la respuesta correspondiente del modelo, la cual se guarda en un fichero con el identificador del programa analizado. Además, se registra en un archivo de texto el tiempo que tarda cada solicitud en procesarse y generar una respuesta por parte del modelo, así como el tiempo total requerido para los 63 programas.

Repitiendo este proceso para cada modelo, según el tipo de programa (*Wallet o Voting*) y el tipo de *prompt* (básico o complejo). Se han obtenido 16 muestras y un total de 1008 respuestas analizadas.

3.3.3. Criterios de evaluación y metodología del análisis

Para analizar las respuestas de los modelos, se ha usado la siguiente tabla:

ID Program	Vuln_1_Real	Vuln_X_Real	Vuln_1_Predicted	Vuln_X_Predcited
1	3	0	1	2
X	0	2	0	2

Cuadro 14: Tabla ejemplo usada para el análisis de las respuestas

Se trata de una tabla varias columnas, la primera de ellas contiene el identificador de cada programa. A esta le siguen otras con el nombre de cada vulnerabilidad más el sufijo **real**; estas columnas pueden tener valores de 0 a 10 (no en todo los casos, depende de la vulnerabilidad) y representan si el programa presenta esa vulnerabilidad y cuántas veces. Además, existen otra serie de columnas con las mismas vulnerabilidades representadas pero con el prefijo **predicted**. Estas columnas indican si las respuestas de los modelos han detectado la vulnerabilidad, cuántas veces lo hicieron o si no la detectaron en absoluto.

Para evaluar si realmente se detecta una vulnerabilidad, se siguen los siguientes criterios:

- **Mención en la función:**

Si la vulnerabilidad se menciona explícitamente en una función, o se resalta que falta un control.

■ Referencia global:

Si se menciona la vulnerabilidad de manera general pero de forma explícita.

En general, todo lo que fuera ambiguo se ignoraría. En el caso del *dataset* para el programa *Wallet*, la detección de vulnerabilidades es relativamente sencilla, ya que se trata de vulnerabilidades básicas y la lógica del programa es simple. Sin embargo, para el caso de los programas *Voting*, la situación es más compleja, ya que la lógica es más difícil de analizar y las vulnerabilidades a detectar no son tan evidentes. Por lo que, hay que tener en cuentas unos casos especiales:

En la vulnerabilidad ***Bump vulnerable***, se da por acertada cuando se mencione al menos uno de los siguientes puntos:

- El *bump* es proporcionado por el usuario, lo cual representa un riesgo.
- El *bump* no se verifica previamente.
- La función *create_program_address* no es confiable.
- No controlar el *bump* implica un riesgo de seguridad.

En la vulnerabilidad ***load_function_at***, se da por acertada cuando se mencione:

- La función *load_function_at* no es segura o está deprecada.
- Usar la cuenta *sysvar instruction* no es seguro para verificar el orden de las operaciones.
- La cuenta *sysvar instruction* es fácilmente manipulable o susceptible a fallos.

3.4. Fase 4: Análisis de los resultados

Como ya se ha mencionado previamente, en el presente trabajo se han analizado 4 grandes modelos de lenguaje, y más concretamente, su capacidad de detectar vulnerabilidades en los 2 tipos de programas definidos anteriormente mediante el uso de 2 *prompts* diferentes. En este apartado, se expondrá la comparación de los modelos y *prompts* entre sí. Más adelante, se discutirán las implicaciones de estos resultados para la programación segura de programas en la *blockchain* Solana.

3.4.1. Capacidad de detectar la presencia de vulnerabilidades

En esta sección se analizará la capacidad de los modelos y los *prompts* de detectar la presencia de las vulnerabilidades en el código. Como ya se ha establecido previamente, un programa puede presentar la misma vulnerabilidad en más de una función, sin embargo, en esta parte del análisis, la repetición de la misma vulnerabilidad no se tendrá en cuenta. En otras palabras, si un programa tiene la vulnerabilidad “A” x veces, no se apunta el número de veces que aparece, únicamente que está presente. Igualmente, no interesa en cuántas funciones el modelo detecta la presencia de la vulnerabilidad “A”, únicamente interesa si ha detectado su presencia en al menos una. Por lo tanto, si un programa tiene 2 veces una vulnerabilidad de *Underflow* y el modelo únicamente menciona una de sus apariencias, se considera que ha detectado correctamente la presencia de la vulnerabilidad.

Por lo tanto, para cada vulnerabilidad se crean dos vectores: Uno que señala la presencia o ausencia de la vulnerabilidad en los programas y otro que señala si el modelo en cuestión la había detectado o no. Podemos utilizar estos dos vectores para crear una matriz de confusión.

En el caso concreto representado en la imagen, es decir, el modelo gpt-nano-4.1 con el *prompt* complejo analizando la vulnerabilidad donde se omite el control de verificación del *PDA* de una cuenta en el programa *Wallet*, podemos observar 14 detecciones correctas de la vulnerabilidad (verdaderos positivos), 28 detecciones correctas de un programa sin la vulnerabilidad (verdaderos negativos), pero también 2 falsos negativos y 19 falsos positivos. A partir de estos datos, podemos calcular varias métricas que nos permiten diagnosticar la calidad de la clasificación. En este trabajo, se empleará la precisión, la sensibilidad y la especificidad.

	PDACheckReal	
PDACheckPredicted	TRUE	FALSE
TRUE	14	2
FALSE	19	28

Figura 8: Matriz de confusión para *PDA check* en un programa *Wallet*

La precisión sería simplemente:

$$precision = \frac{\text{número de predicciones correctas}}{\text{número de predicciones}}$$

La sensibilidad se calcularía de la siguiente manera:

$$sensibilidad = \frac{\text{número de verdaderos positivos}}{\text{número de verdaderos positivos} + \text{número de falsos negativos}}$$

En otras palabras, la sensibilidad comunica qué porcentaje de programas vulnerables han sido identificados identificados como vulnerables.

Finalmente, la especificidad se calcularía de la siguiente manera:

$$especificidad = \frac{\text{número de verdaderos negativos}}{\text{número de verdaderos negativos} + \text{número de falsos positivos}}$$

En otras palabras, la sensibilidad comunica qué porcentaje de programas seguros han sido identificados como seguros.

En el caso de la matriz de confusión anterior, su precisión equivaldría a 66.6 %, la sensibilidad a 87,5 % y la especificidad a 59,57 %.

De la misma manera, podemos evaluar el rendimiento de todos los modelos y *prompts* en cada una de las vulnerabilidades. Comenzamos por el programa *Wallet*, para facilitar la comparación, las gráficas correspondientes a una misma vulnerabilidad se han colocado adyacentes, mostrando los resultados obtenidos con diferentes *prompts*.

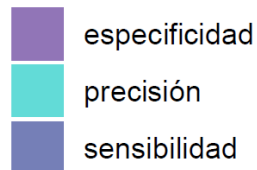
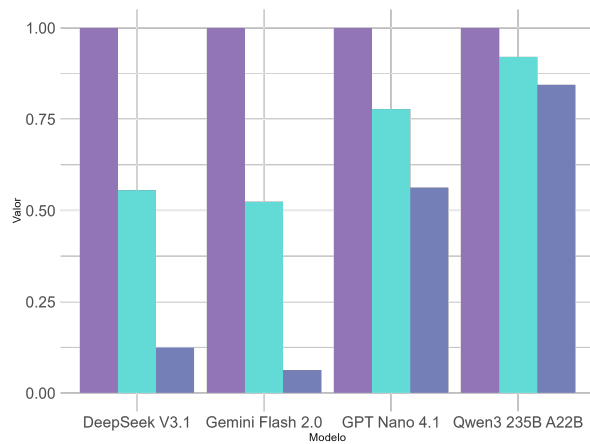
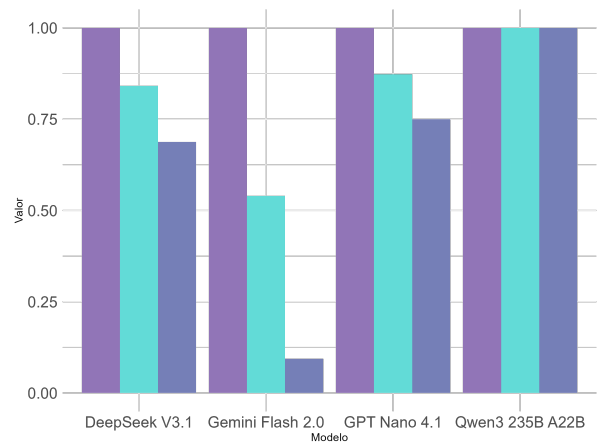


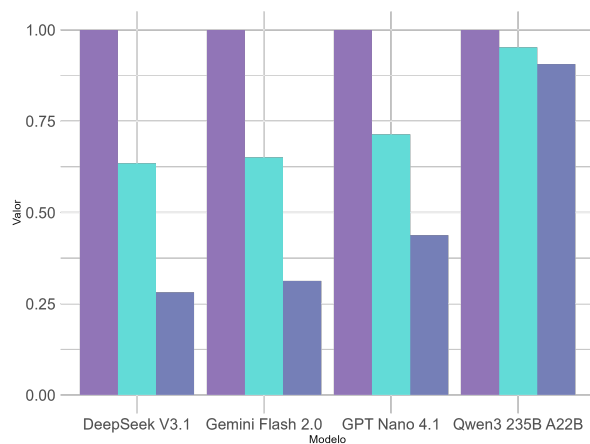
Figura 9: Leyenda - Métricas de la matriz de confusión



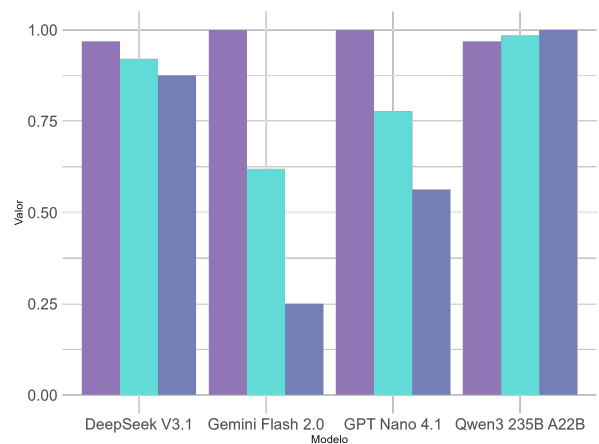
(a) Owner check con prompt básico



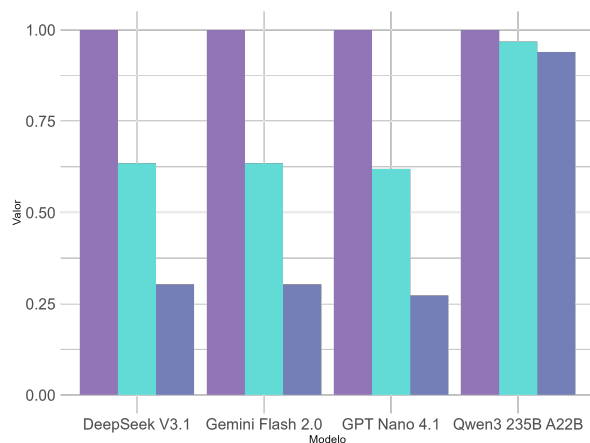
(b) Owner check con prompt complejo



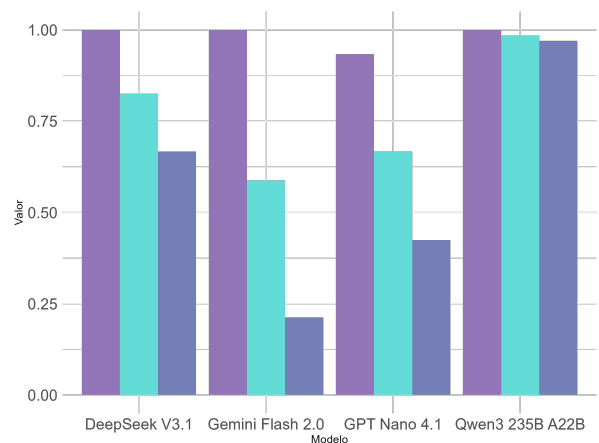
(c) Signature check con prompt básico



(d) Signature check con prompt complejo

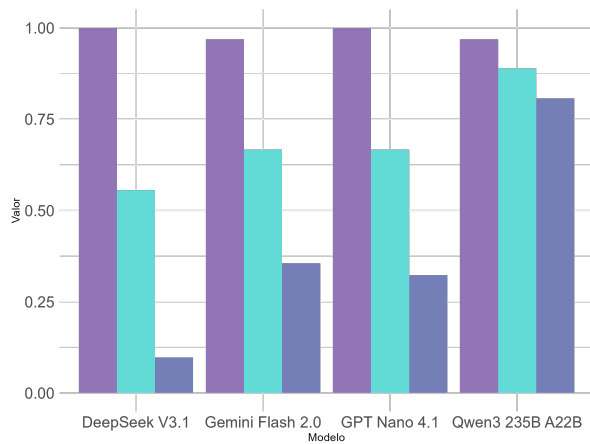


(e) PDA check con prompt básico

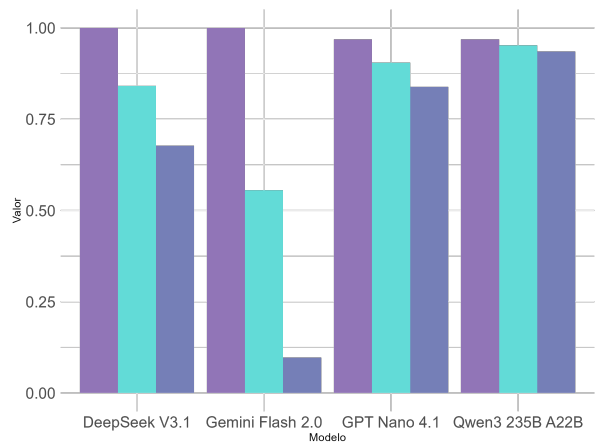


(f) PDA check con prompt complejo

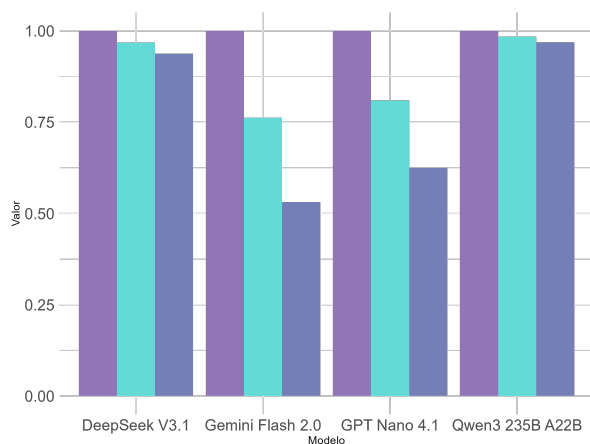
Figura 10: Análisis de precisión, sensibilidad y especificidad



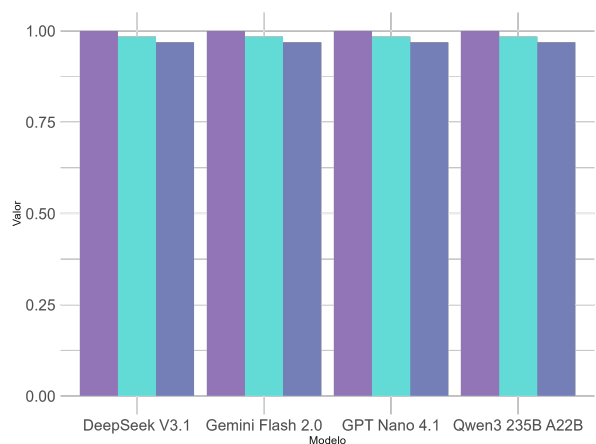
(a) Account initialization check con prompt básico



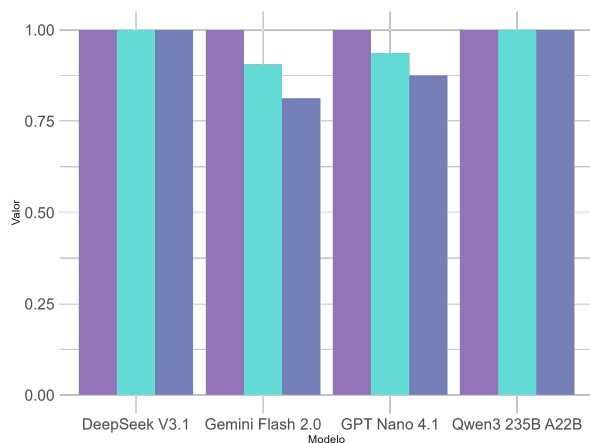
(b) Account initialization check con prompt complejo



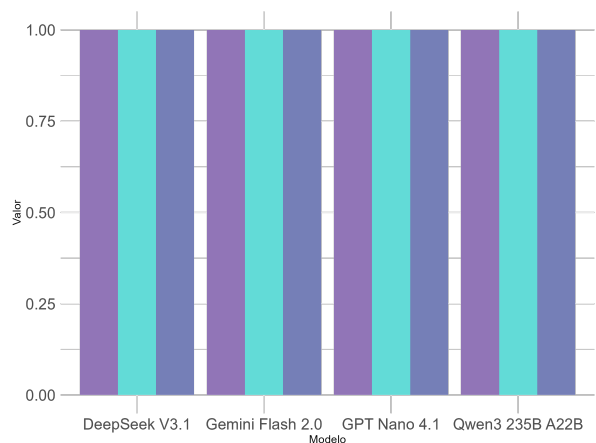
(c) Overflow check con prompt básico



(d) Overflow check con prompt complejo



(e) Underflow check con prompt básico



(f) Underflow check con prompt complejo

Figura 11: Análisis de precisión, sensibilidad y especificidad

En términos generales, podemos observar que todos los modelos, independientemente del

prompt utilizado y para la mayoría de las vulnerabilidades, presentan una especificidad muy alta, alcanzando incluso el 100 % en la detección de programas sin fallos. Este dato permite concluir que, en general, ninguno de los modelos tiende a “inventar” vulnerabilidades cuando no existen.

Al analizar la precisión en la detección de la ausencia de controles para *Owner*, *Signature* y *PDA*, se observa que, en el caso de GPT y Gemini, no se mejoran los resultados al usar un *prompt* complejo en comparación con los valores obtenidos con el *prompt* básico, mostrando solo una leve mejora. En contraste, el modelo DeepSeek presenta una diferencia notable según el tipo de *prompt* utilizado, evidenciando mejoras claras en su desempeño cuando se trabaja con instrucciones más complejas.

Por otro lado, en el modelo de código abierto Qwen3, mantiene un nivel de precisión bastante alto tanto con el *prompt* básico como con el complejo, mostrando únicamente una mejora mínima al aumentar la complejidad del *prompt*.

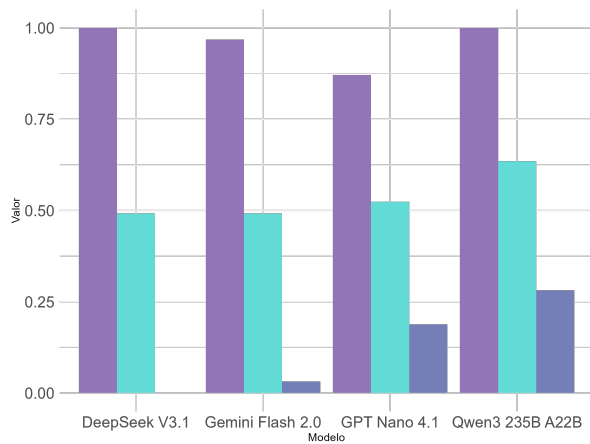
En cuanto a la sensibilidad, se observa que el modelo Gemini es, sin duda, el menos efectivo a la hora de detectar programas vulnerables, tanto con *prompts* básicos como complejos. En contraste, los demás modelos muestran una mejora notable en su desempeño cuando se utilizan *prompts* más específicos, lo que refleja una mayor capacidad para identificar correctamente los programas con vulnerabilidades.

Con respecto al control *Account Initialize*, se observa que todos los modelos mejoran sus métricas al utilizar un *prompt* más complejo. Sin embargo, el modelo Gemini sigue mostrando un desempeño inferior en comparación con los demás, manteniéndose como el menos efectivo.

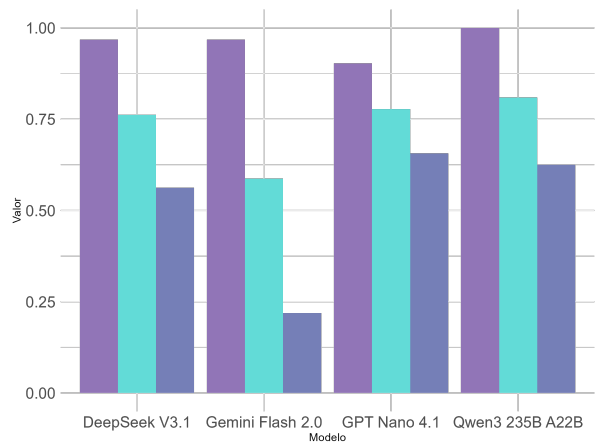
En el caso de las vulnerabilidades de *Overflow* y *Underflow*, se observa un escenario prácticamente ideal, ya que todos los modelos presentan valores muy altos en todas las métricas evaluadas. Entre ambos tipos, *Underflow* muestra los mejores resultados en general.

En conclusión, para este tipo de programa con una lógica simple y controles que son fundamentales, los modelos GPT y Gemini tienden a obtener un desempeño inferior en términos de precisión y sensibilidad en comparación con los modelos DeepSeek y Qwen3, lo que indica una menor efectividad a la hora de identificar correctamente la existencia de estas vulnerabilidades.

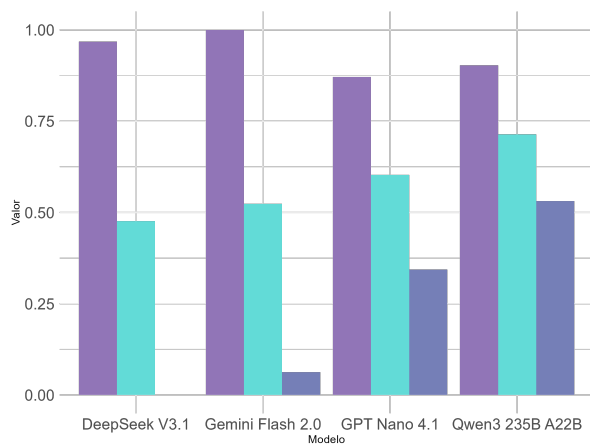
Seguimos con los programas del programa *Voting*:



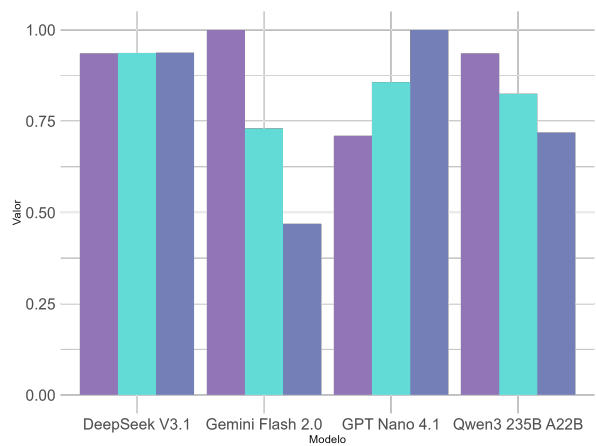
(a) Bump vulnerable con prompt básico



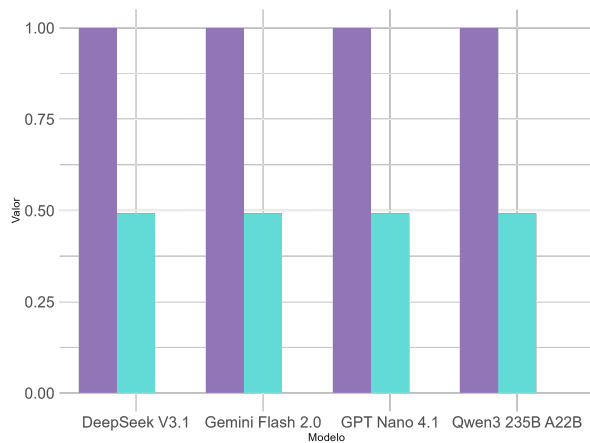
(b) Bump vulnerable con prompt complejo



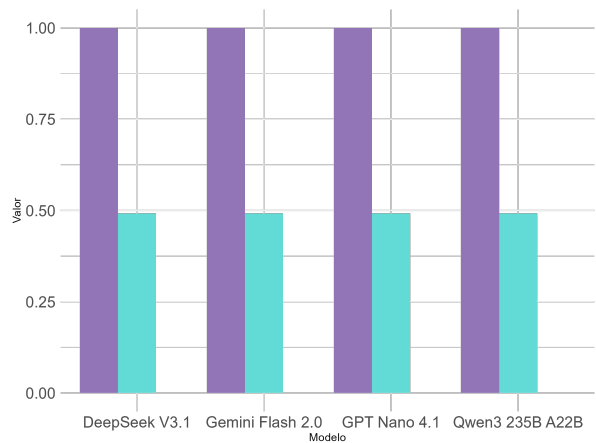
(c) Admin check con prompt básico



(d) Admin check con prompt complejo

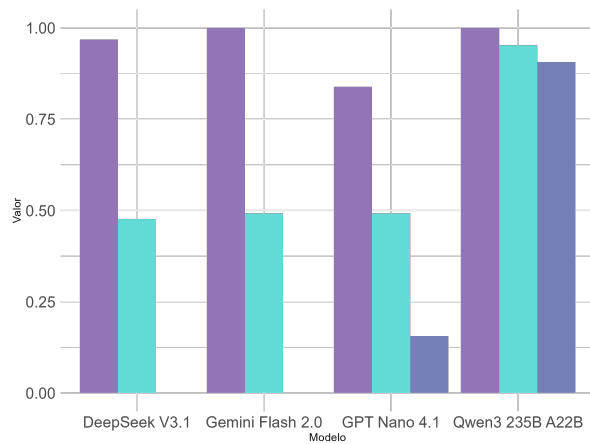


(e) Account confusion con prompt básico

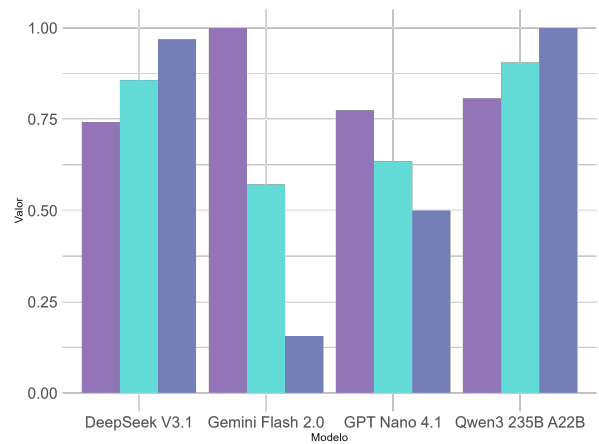


(f) Account confusion con prompt complejo

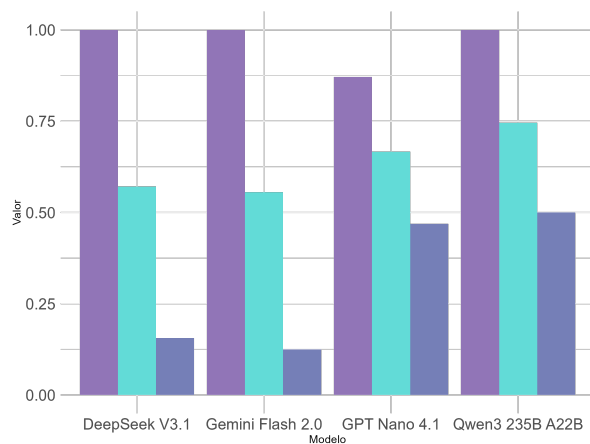
Figura 12: Análisis de precisión, sensibilidad y especificidad



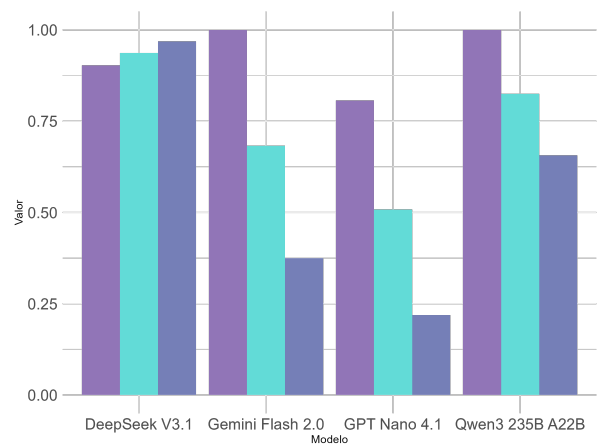
(a) CPI ID check con prompt básico



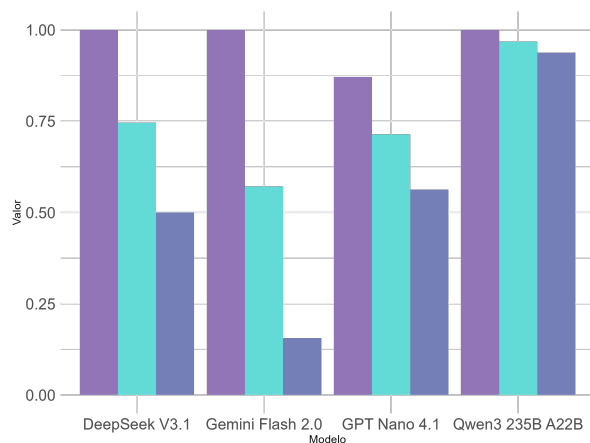
(b) CPI ID check con prompt complejo



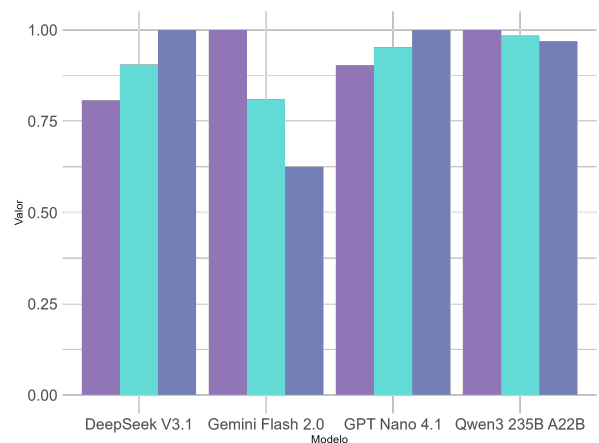
(c) Overflow check con prompt básico



(d) Overflow check con prompt complejo



(e) Underflow check con prompt básico



(f) Underflow check con prompt complejo

Figura 13: Análisis de precisión, sensibilidad y especificidad

En el caso de las vulnerabilidades de tipo *Bump* y *Admin Check*, se observa que, en general,

todos los modelos presentan precisión baja, lo que indica que tienen dificultades para predecir correctamente qué programas son vulnerables. La sensibilidad tampoco es alta, siendo en algunos casos incluso nula, como ocurre con DeepSeek. Sin embargo, al utilizar un *prompt* más complejo, aunque algunos valores sigan siendo bajos, se nota una mejor en comparación con los resultados obtenidos con el *prompt* básico. Esto refleja que los modelos se benefician de instrucciones más detalladas para identificar este tipo de vulnerabilidades.

En el caso de las vulnerabilidades relacionadas con *Account Confusion*, los modelos no lograron detectar ningún caso. Esto puede deberse a que se trata de un error bastante abstracto y lógico, y probablemente se centraban en vulnerabilidades más evidentes. Para mejorar la detección de este tipo de fallo, quizás habría sido necesario indicar explícitamente en el *prompt* que se buscará esta vulnerabilidad o proporcionar una pista adicional, como el uso del campo tipo en la estructura de la cuenta. Sin embargo, esta estrategia tendría poca aplicabilidad en un entorno real, ya que normalmente no se incluiría esa pista de manera explícita.

En el caso de las vulnerabilidades *CPI*, *Overflow* y *Underflow*, se observa que, con el *prompt* básico, los modelos presentan valores bajos en las métricas evaluadas. Sin embargo, gracias al uso de un *prompt* más específico, se logra una mejora en su desempeño, aumentando notablemente sus medidas de precisión y sensibilidad.

A partir de estos datos, podemos discernir que, en general, para todos los programas, al introducir un *prompt* complejo, los modelos mejoraron notablemente la manera de generar sus respuestas: dejaron de limitarse a describir lo que encontraban en el código y comenzaron a ofrecer análisis más relevantes centrados en las vulnerabilidades. Esto sobre todo sucede con los modelos GPT, Gemini y DeepSeek. La única excepción fue el modelo de código abierto Qwen3, que incluso con el *prompt* básico aportaba información de valor orientada a la detección de fallos. Los problemas más fáciles de detectar para los modelos son las vulnerabilidades de *Underflow*, seguidas de las de *Overflow*. Esto resulta sorprendente, ya que inicialmente podría pensarse que los controles más básicos y fundamentales al escribir un programa en Solana, como *Owner*, *Signature* y *PDA*, serían los más detectados. Sin embargo, según los datos, esto no puede darse por sentado.

Podemos observar que la especificidad en los programas de tipo *Wallet* se mantiene siempre en valores muy altos. En cambio, para los de tipo *Voting*, estos valores tienden a ser menores. Aunque no son malos, son inferiores en comparación con los de *Wallet*, lo que

indica que la complejidad del programa influye en la capacidad de los modelos para determinar correctamente la presencia o ausencia de vulnerabilidades. Finalmente, se observa que los modelos GPT y Gemini son los menos efectivos para estas tareas, debido a las bajas métricas obtenidas. En contraste, DeepSeek y Qwen3 destacan por sus altos valores en las diferentes medidas, consolidándose como los modelos más confiables.

Como conclusión, es recomendable elegir un modelo más específico y orientado al análisis de código. No sería suficiente optar por modelos rápidos o económicos, como GPT Nano o Gemini Flash, que no ofrecen la precisión necesaria. Qwen3 representa una buena opción, ya que combina especialización en código con eficiencia económica.

Igualmente, en el caso de tener que utilizar modelos como GPT o Gemini, quizás se podría mejorar su eficiencia en la detección de vulnerabilidades el crear un *prompt* más complejo y específico, en el que además se detallara la lógica del programa en cuestión. De lo contrario, los modelos tienden a hacer suposiciones e incluso inventarse lógica. Especificar claramente esta información podría ayudarles a enfocarse en la búsqueda de vulnerabilidades en lugar de proponer correcciones innecesarias o directamente incorrectas sobre la lógica del programa.

3.4.2. Capacidad de detección del número correcto de vulnerabilidades

En este apartado, se examinará la cantidad de vulnerabilidades que detectan los diferentes modelos y *prompts*. Los datos se examinarán mediante el error medio absoluto (MAE), raíz del error cuadrático medio (RMSE) y el sesgo.

El error medio absoluto se utilizará para ver la distancia media entre el número real de vulnerabilidades y el número predicho. Se puede calcular utilizando la siguiente fórmula:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Donde n es el número de casos totales, y es el número de errores real e $\hat{y}$ es el número de errores predicho.

La raíz del error cuadrático medio se utiliza sobre todo para penalizar los errores más grandes en los que la predicción se aleja en gran medida del número real. Aunque a nivel descriptivo no sea la medida más relevante, se utiliza frecuentemente como función de pérdida y por lo tanto, podría emplearse para tunear un modelo especializado. Podemos conseguir su

valor utilizando la siguiente fórmula:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Otra vez, n es el número de casos totales, y es el número de errores real e $\hat{y}$ es el número de errores predicho

Finalmente, el sesgo se empleará para ver la dirección del error, es decir, si el modelo predice más vulnerabilidades de las que hay (sesgo negativo) o menos de las que hay (sesgo positivo).

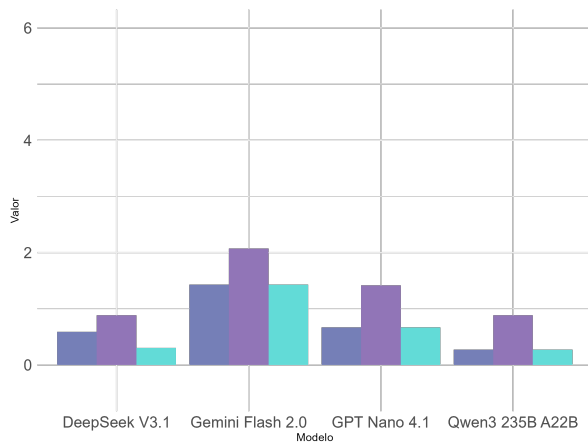
$$\text{Sesgo} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)$$

Igualmente que en los casos anteriores, n es el número de casos totales, y es el número de errores real e $\hat{y}$ es el número de errores predicho.

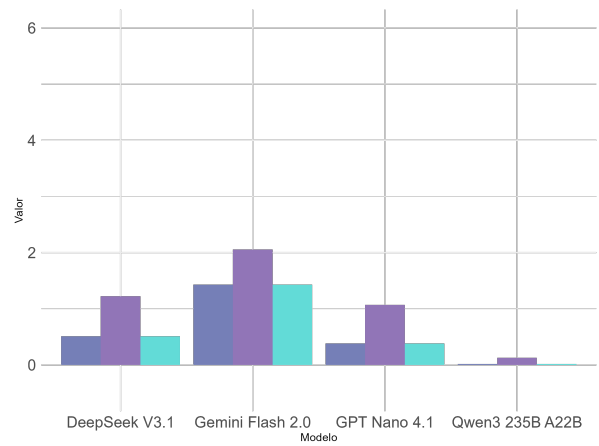


Figura 14: Leyenda - Métricas de error

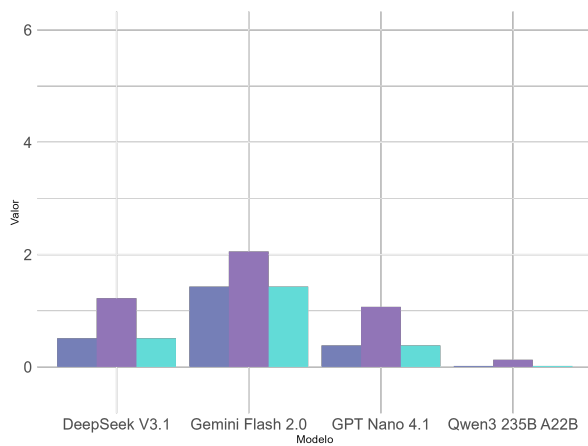
Podemos examinar los errores en los programas *Wallet*:



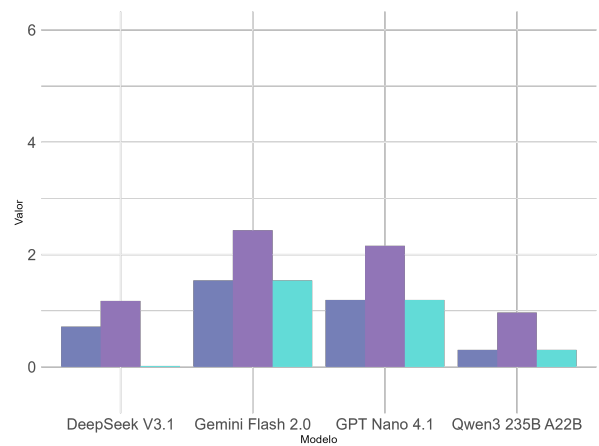
(a) Owner check con prompt básico



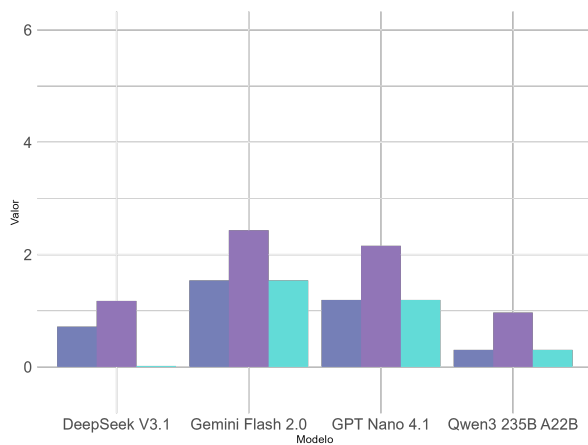
(b) Owner check con prompt básico



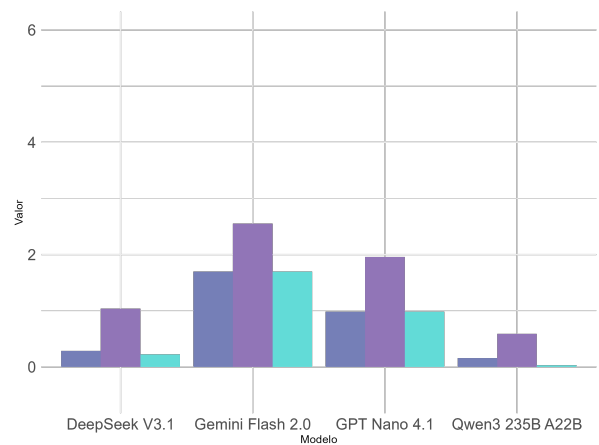
(c) Signature check con prompt básico



(d) Signature check con prompt complejo

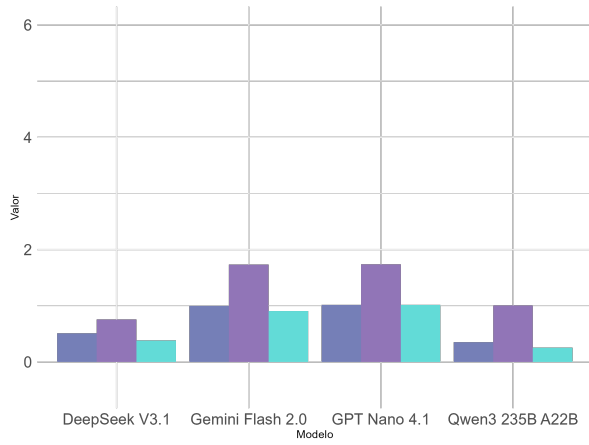


(e) PDA check con prompt básico

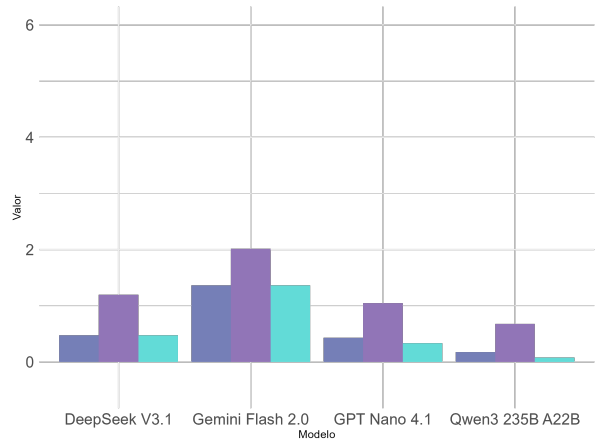


(f) PDA check con prompt complejo

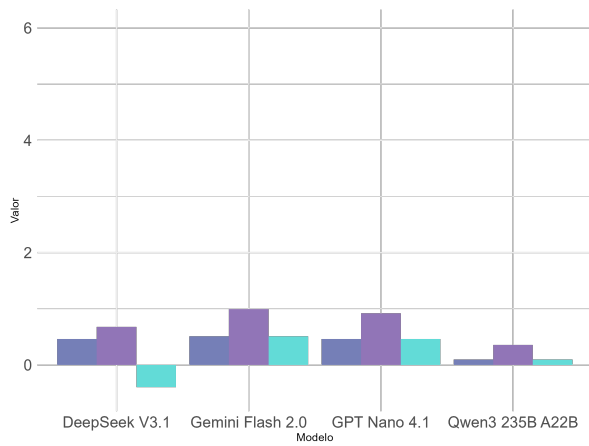
Figura 15: Análisis de MAE, RMSE y sesgo



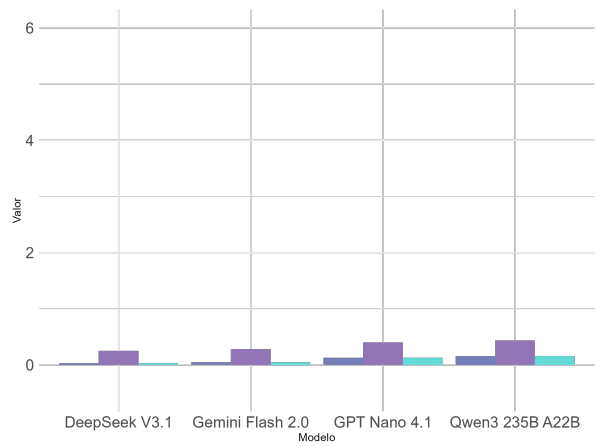
(a) Account initialization check con prompt básico



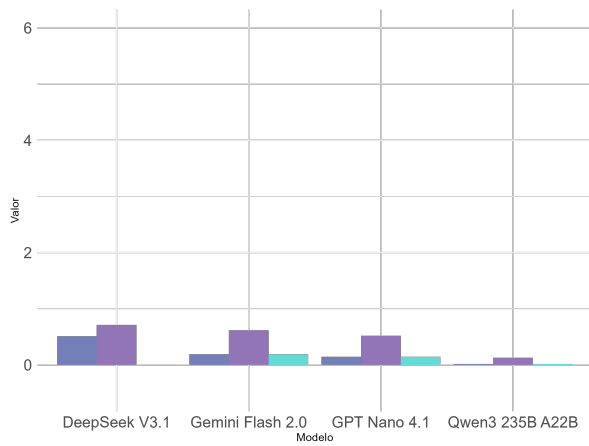
(b) Account initialization check con prompt complejo



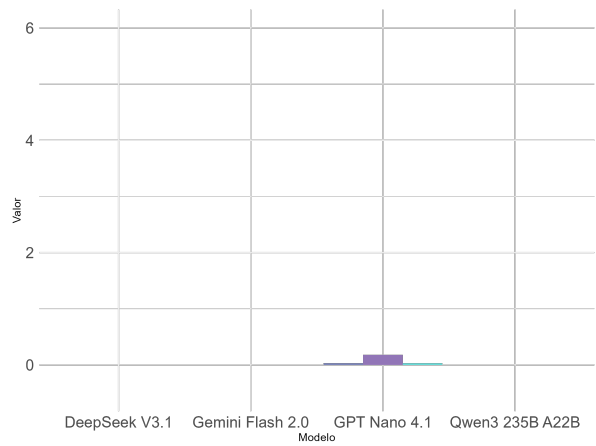
(c) Overflow check con prompt básico



(d) Overflow check con prompt complejo



(e) Underflow check con prompt básico



(f) Underflow check con prompt complejo

Figura 16: Análisis de MAE, RMSE y sesgo

Analizando el caso de los *prompts* básicos, observamos que las vulnerabilidades *Underflow*,

Owner y *Overflow* son detectadas con valores bastante buenos por la mayoría de los modelos. Tanto el sesgo como el MAE se mantienen por debajo de uno, lo cual indica que, en general, se logra identificar la mayoría de las vulnerabilidades presentes en cada función. Cabe destacar que el número real de vulnerabilidades para cada categoría es 2, 3 y 2, respectivamente.

Asimismo, al introducir un *prompt* más complejo, la mayoría de los modelos disminuye sus errores de forma notable. La excepción se presenta en el modelo Gemini para el caso de *Owner*, donde se evidencia un error más amplio y, además, no se observa ninguna mejora frente a la introducción del *prompt* más elaborado.

Para el caso de la falta de control de la firma (*signature*) y de *PDA*, se puede afirmar que los modelos GPT y Gemini presentan el peor desempeño, y no muestran mejoras relevantes en contraste con Qwen3 que además muestra mejoras al utilizar un *prompt* más complejo. No obstante, en términos generales, todos los modelos muestran un rendimiento aceptable, dado que el MAE y el sesgo suele estar por debajo de 2, lo que indica que son capaces de detectar la mayoría de las vulnerabilidades en las funciones, cuyo valor real para ambas es 4 en este tipo de programa. Sin embargo, se observa que DeepSeek presenta comportamientos irregulares: en varias ocasiones, al utilizar un *prompt* más complejo, su error aumenta en lugar de disminuir.

En cuanto a *Account Initialize*, los peores desempeños corresponden a Gemini y GPT. Gemini incluso empeora en las pruebas con el *prompt* complejo, mientras que GPT corrige su error ligeramente. DeepSeek tiende a empeorar con el *prompt* complejo, en tanto que Qwen3 muestra una mejora clara.

En general, no se observa que la especificación del *prompt* garantice siempre una disminución de los errores. En el caso del modelo Qwen3, sí se logra una mejora notable; sin embargo, con Gemini no ocurre lo mismo. Para DeepSeek puede haber mejoras, pero en algunos casos incluso empeora, algo que también sucede con el modelo GPT. Esto demuestra que no se puede confiar únicamente en que un *prompt* más detallado sea suficiente para asegurar que el modelo detecte correctamente todas las vulnerabilidades presentes en un programa.

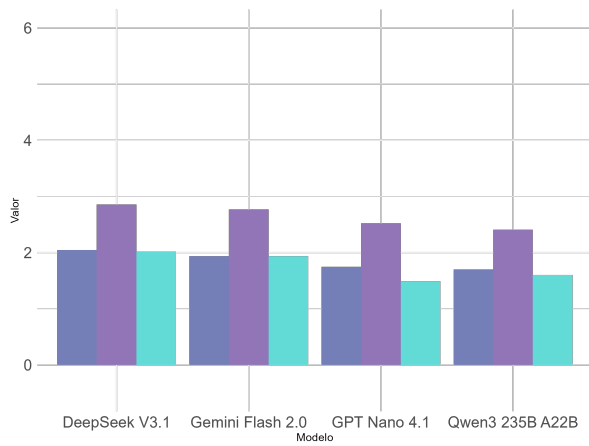
Y de la misma manera, para los programas *Voting*:



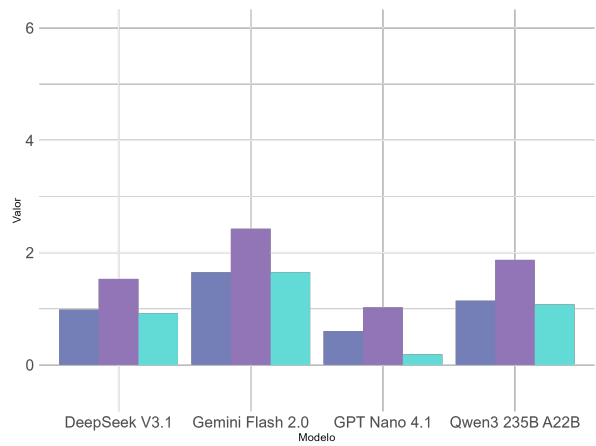
(a) Bump vulnerable con prompt básico



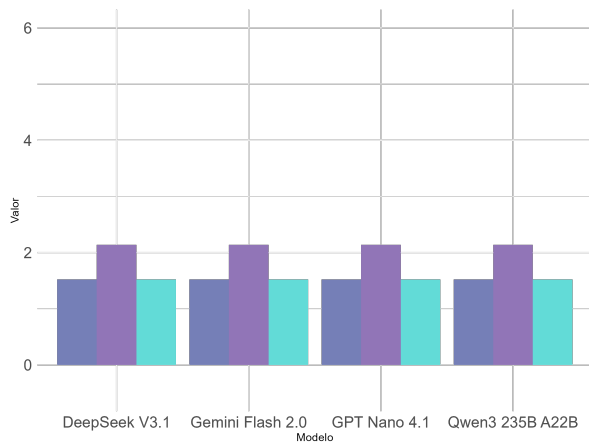
(b) Bump vulnerable con prompt complejo



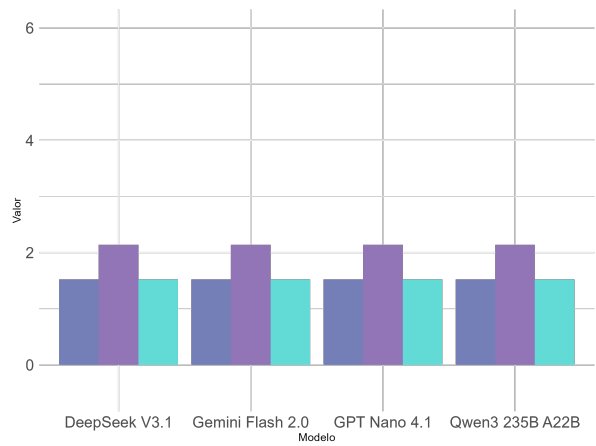
(c) Admin check con prompt básico



(d) Admin check con prompt complejo

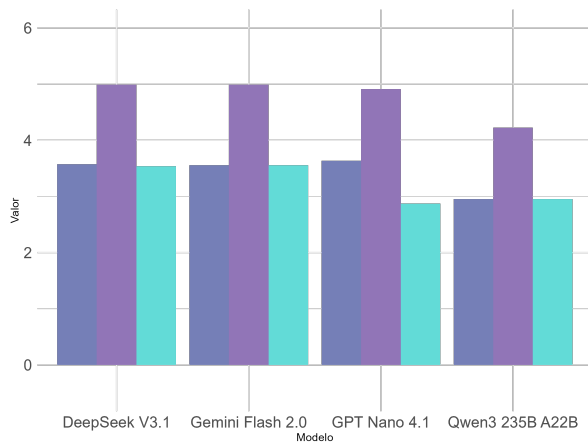


(e) Account confusion con prompt básico

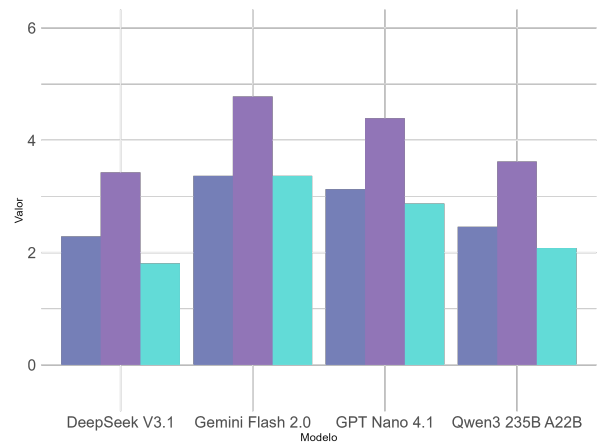


(f) Account confusion con prompt complejo

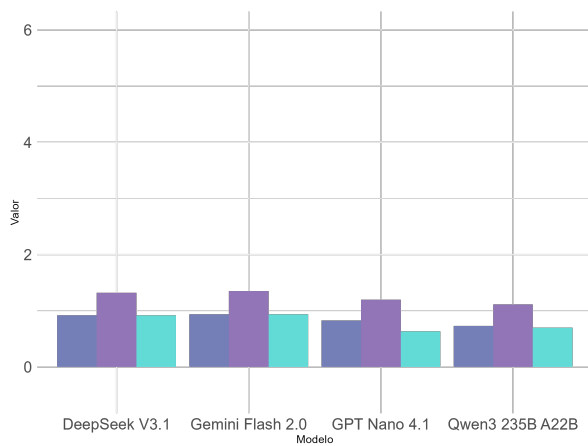
Figura 17: Análisis de MAE, RMSE y sesgo



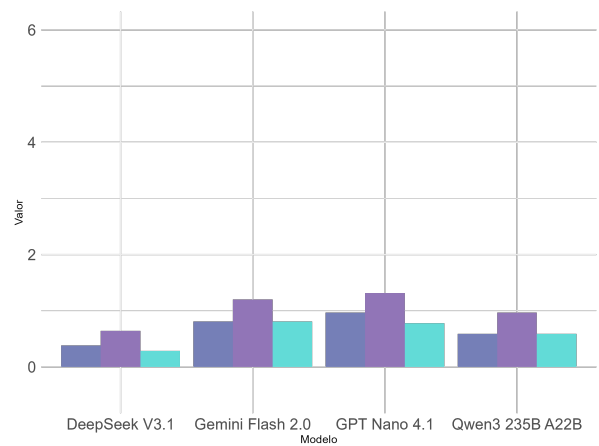
(a) CPI ID check con prompt básico



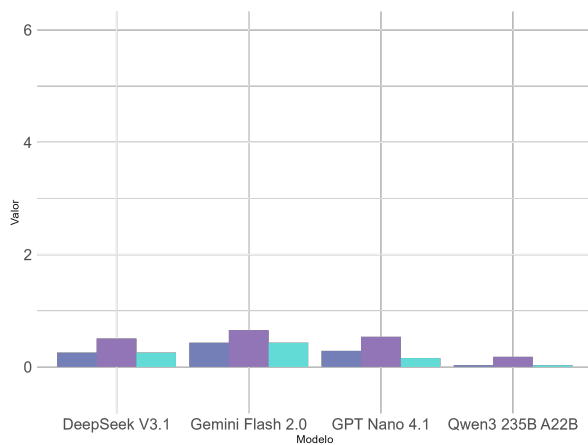
(b) CPI ID check con prompt complejo



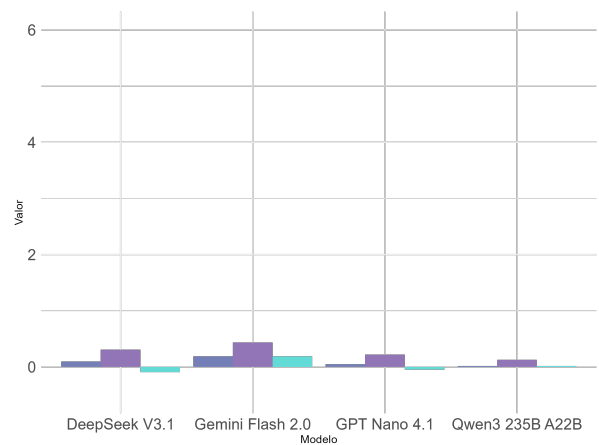
(c) Overflow check con prompt básico



(d) Overflow check con prompt complejo



(e) Underflow check con prompt básico



(f) Underflow check con prompt complejo

Figura 18: Análisis de MAE, RMSE y sesgo

En el caso del programa *Voting*, por lo general se detectan sin dificultad los problemas de

Underflow, mostrando además mejoras al introducir un *prompt* más complejo. En cuanto a *Overflow*, se observa que los errores son mayores que en el programa *Wallet*; y al introducir un *prompt* complejo, la mejora es mínima y, en todo caso, se aprecia únicamente en el modelo DeepSeek.

Las vulnerabilidades de *bump* y *CPI* son particulares, ya que sus valores reales son 8 y 7, respectivamente. En el caso de *bump*, el sesgo muestra que, con el *prompt* básico, en la mayoría de los casos se omiten alrededor de 4 vulnerabilidades. Al introducir un *prompt* más complejo, esta métrica disminuye levemente, acercándose a 3. En este escenario, se observa que GPT obtiene mejores resultados, pues mantiene el sesgo por debajo de 3.

Para *CPI* ocurre una situación similar: en promedio, los modelos tienden a omitir aproximadamente 3 vulnerabilidades, lo cual representa un desempeño aceptable considerando el valor real. Además, al utilizar un *prompt* más elaborado, la mayoría de los modelos reducen sus errores, siendo DeepSeek y Qwen3 los que muestran la mayor mejora.

Al analizar la vulnerabilidad de *Admin Check*, se observa que los modelos tienden a omitir en promedio 2 instancias, considerando que el valor real es 4 para este control. No obstante, al introducir un *prompt* más complejo, todos los modelos mejoran su rendimiento, con la excepción de Gemini, que no muestra avances.

En cuanto a la vulnerabilidad de *Account Confusion*, esta se mantiene sin detección, ya que no se identificó ningún caso en los resultados obtenidos.

En resumen, no se observan grandes diferencias entre los modelos al utilizar *prompts* básicos, aunque sí se aprecia una ligera mejora al introducir un *prompt* complejo, con la excepción de Gemini, que se mantiene prácticamente igual. Un aspecto interesante es que, mientras en los análisis de las métricas de la matriz de confusión GPT aparecía como un modelo débil para la detección de casos, en este escenario muestra un rendimiento positivo cuando se trata de contar las vulnerabilidades presentes en las funciones, por lo que podemos pensar que, o bien detecta la mayoría de ellas, o no detecta ninguna. En general, logran identificar un número aceptable de vulnerabilidades, acercándose a la mitad del número real presente en las funciones.

Igualmente, es importante tener en cuenta que, en muchos modelos, incluso al introducir un *prompt* más complejo, los errores pueden llegar a empeorar. Esto permite inferir que el hecho de utilizar un *prompt* más detallado no es suficiente para garantizar que el modelo

detecte correctamente todas las vulnerabilidades presentes en un programa.

3.4.3. Detecciones de la función deprecada *load_function_at* y el uso de la cuenta *sysvar instructions*

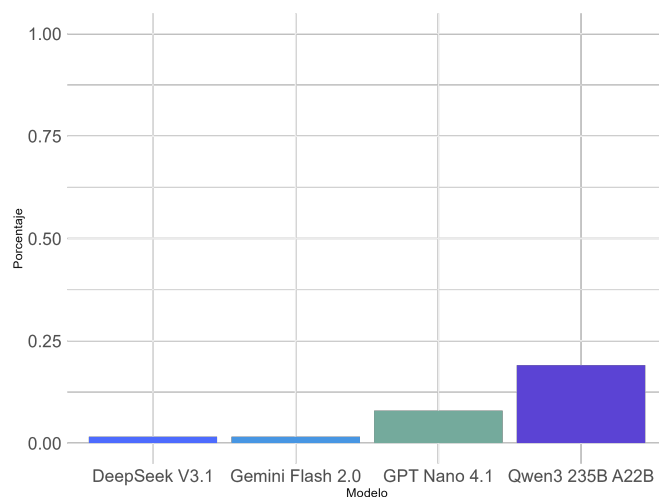


Figura 19: Porcentaje de aciertos con prompt básico

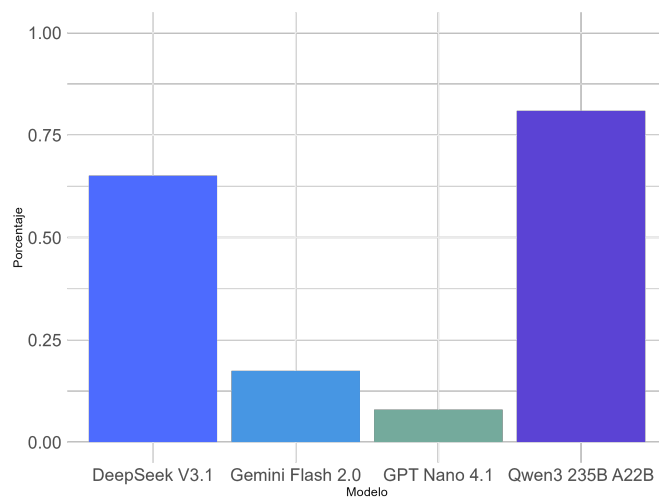


Figura 20: Porcentaje de aciertos con prompt complejo

Podemos observar que, en general, los modelos tienden a acertar poco en el caso del *prompt* básico, siendo únicamente Qwen3 y GPT Nano los que muestran un desempeño relativamente superior.

Sin embargo, al introducir el *prompt* complejo, la detección mejora significativamente para Deepseek y Qwen3, mientras que GPT Nano se mantiene prácticamente constante y Gemini presenta un aumento leve en su desempeño.

Conclusiones y Líneas Futuras

En este trabajo se buscó explorar el alcance de diferentes *LLMs*, con capacidades variadas, para analizar su desempeño frente a un problema concreto, el análisis de vulnerabilidades de *Programs* escritos en Rust a ser desplegados en la *blockchain* de Solana, un área que hasta ahora ha sido poco explorada. El objetivo no solo fue comparar sus respuestas frente a otros modelos, sino también evaluar si, al modificar las instrucciones que se les proporcionan, se puede mejorar su capacidad para realizar esta tarea.

Se utilizaron distintos modelos, todos ellos con la característica común de ser rápidos, eficientes, ser de conocimiento general y tener un bajo coste económico, aunque algunos mostraban diferencias en *benchmarks* relacionados con la resolución de problemas de software o con conocimientos de programación. Entre ellos, los menos potentes en estas áreas fueron GPT y Gemini, mientras que Deepseek y Qwen3 obtuvieron mejores resultados en dichas evaluaciones.

Para responder a esta pregunta se recrearon dos programas diferentes, con niveles de lógica y vulnerabilidades presentes distintas. Para realizar las pruebas, a cada modelo se le proporcionó dos tipos de *prompts*: uno básico, donde no se especificaban instrucciones y la tarea era general; y otro complejo, donde la tarea se estructuraba mejor y se resaltaban las instrucciones de manera más explícita.

Se analizaron los resultados a fin de determinar cuántas predicciones correctas realizó el modelo (precisión), cuántos programas vulnerables fueron identificados correctamente (sensibilidad) y qué programas se identificaron como seguros (especificidad). Además, se realizó un estudio de los errores cometidos al contar cuántas vulnerabilidades de un mismo tipo existían en un programa, lo que permitió determinar si el modelo tiende a detectar más o

menos vulnerabilidades de las que realmente hay (sesgo), así como la distancia media del error respecto al valor real (MAE) y el error cuadrático medio (RMSE).

Adicionalmente, se exploró una función que fue clave en un ataque real conocido como *Workmoke Attack*, muy relevante en el ecosistema *blockchain*. Se evaluó si, en casos específicos, los modelos eran capaces de detectar la presencia de esta función obsoleta, comparando los resultados obtenidos con *prompts* básicos frente a complejos.

Finalmente, una vez evaluados todos los resultados, se concluye que el modelo menos eficiente para el análisis de *Programs* en Solana dentro del presente estudio sería Gemini Flash 2.0, ya que generalmente presentaba más errores y detectaba menos programas con vulnerabilidades. Incluso en muchos casos, la introducción del *prompt* complejo apenas mejoraba sus resultados.

En segundo lugar se encuentra GPT Nano 4.1, que mostró un comportamiento similar al de Gemini. Sin embargo, en muchos casos un *prompt* complejo permitió enfocar mejor su análisis y aumentar su rendimiento en la detección de vulnerabilidades.

A continuación, Deepseek V3.1 demostró un desempeño superior, con buenas detecciones incluso usando el *prompt* básico, y con mejoras notables al aplicar el *prompt* complejo.

Por último, el modelo que presentó mejores resultados fue Qwen3. Este modelo mostró valores altos en todas las métricas, con precisión y sensibilidad elevadas, y errores muy bajos desde las primeras pruebas con el *prompt* básico. Sus respuestas eran consistentes y mejor estructuradas que las de los demás modelos, que a menudo solo se limitaban a describir lo que analizaban cuando se usaba un *prompt* básico. Además, el rendimiento de Qwen3 mejoró aún más con la introducción del *prompt* complejo, consolidándose como el modelo más efectivo para esta tarea.

Con respecto a las vulnerabilidades, se observó que las más fáciles de detectar para todos los modelos son las de tipo aritmético, en contraste con lo que podría esperarse, ya que cabría pensar que tenderían a identificar primero los controles fundamentales de los programas, como la verificación de *Owner*, *Signature* y *PDA*. Una posible explicación es que los errores aritméticos constituyen una vulnerabilidad común en numerosos lenguajes de programación, lo que hace comprensible que los modelos los reconozcan con mayor facilidad.

Para vulnerabilidades más complejas, como la detección de *Bumps* vulnerables, los modelos presentan numerosos errores, acertando con poca frecuencia. Asimismo, se observa que

resulta muy difícil que los modelos identifiquen todas las vulnerabilidades presentes que se repiten en un gran número de funciones, como ocurre con los casos que presentan un *Bump* vulnerable y no se controla el ID durante la ejecución de una invocación a otro programa (*CPI*). También se evidencia que problemas muy abstractos, como los relacionados con la confusión de cuentas, no son correctamente discernidos por los modelos. Cabe mencionar que, respecto a la frecuencia de las vulnerabilidades en un programa, no se puede depender únicamente del *prompt* utilizado, ya que las pruebas muestran claramente que incluir un *prompt* mejorado no siempre influye en cuántas veces los modelos detectan las vulnerabilidades. Este comportamiento se observa, por ejemplo, en Deepseek y GPT Nano, donde en algunos casos los *prompts* mejorados incluso empeoran el rendimiento.

Finalmente, al analizar el porcentaje de aciertos de la vulnerabilidad presente en el *Wormhole Attack*, vemos que la introducción de un *prompt* complejo efectivamente ayuda a detectar mejor la presencia esta función obsoleta o el uso de la cuenta *instruction sysvar* sin ningún tipo de medida de seguridad.

Este proyecto se centró en explorar las capacidades de ciertos modelos de lenguaje frente a casos concretos y específicos. La investigación podría profundizarse mucho más mediante un análisis más detallado, ajustando en los mismos modelos diferentes parámetros como la temperatura, el número máximo de *tokens* de salida o la redacción diferentes *prompts*, para evaluar cómo estas variaciones afectan el desempeño en la detección de vulnerabilidades.

Asimismo, este proyecto podría enriquecerse aún más si se realizaran pruebas similares con modelos más potentes o implementando nuevas técnicas como los *RAGs* (Retrieved Augmented Technology) [63], una metodología que permite obtener resultados más precisos y de mayor calidad ya que los *LLMs* no dependen únicamente del conocimiento paramétrico del modelo, sino que, al procesar un *token* o una consulta, pueden recuperar información relevante desde una base de datos o repositorio externo.

Otra posible línea de trabajo sería explorar *Swarm* [64], una tecnología reciente basada en algoritmos de búsqueda colaborativa. La idea consiste en emplear varios *LLMs*, cada uno especializado en un tema específico, de modo que cada uno se enfoque en una tarea concreta, lo que permitiría alcanzar mayor precisión y calidad en las respuestas.

Referencias

- [1] T. S. BV, *TIOBE Programming Community Index*. dirección: <https://www.tiobe.com/tiobe-index/>.
- [2] D. Yaga, P. Mell, N. Roby y K. Scarfo, “Blockchain Technology Overview,” National Institute of Standards y Technology, inf. téc. NISTIR 8202, 2018. dirección: <https://doi.org/10.6028/NIST.IR.8202>.
- [3] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” SSRN, 2008. dirección: <http://dx.doi.org/10.2139/ssrn.3440802>.
- [4] V. Buterin, “Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform,” Ethereum Foundation, 2013. dirección: https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_Buterin_2014.pdf.
- [5] A. Yakovenko, “Solana: A New Architecture for a High Performance Blockchain v0.8.13,” Solana Labs, 2020. dirección: <https://solana.com/solana-whitepaper.pdf>.
- [6] S. Foundation, *Modelo de cuenta de Solana*. dirección: <https://solana.com/es/docs/core/accounts>.
- [7] Solana Developers, *Program Derived Address (PDA)*. dirección: <https://solana.com/es/docs/core/pda>.
- [8] S. Foundation, *Programas principales (Core Programs) — Documentación de Solana*. dirección: <https://solana.com/es/docs/core/programs>.
- [9] N. Matsakis y A. Turon, *Foreword - The Rust Programming Language*. dirección: <https://doc.rust-lang.org/stable/book/foreword.html>.
- [10] S. Labs, *solana_{program}3,0,0 — Rust API Documentation*. dirección: https://docs.rs/solana-program/latest/solana_program/.
- [11] A. Maintainers, *solana_sdk 3.0.0 — Rust API Documentation*. dirección: https://docs.rs/solana-sdk/latest/solana_sdk/.
- [12] A. C. E. Team, *Solana CLI Tool Suite - Agave Validator Documentation*. dirección: <https://docs.anza.xyz/cli/>.

- [13] S. Foundation, *@solana/web3.js: Solana JavaScript SDK (versión 1.x)*, ver. 1.98.4. dirección: <https://www.npmjs.com/package/@solana/web3.js>.
- [14] A. C. Team, *Anchor Documentation*. dirección: <https://www.anchor-lang.com/docs>.
- [15] S. Gilbert, *Crypto, Web3 and the Metaverse*, 2022. dirección: <https://bennettschool.cam.ac.uk/wp-content/uploads/2022/03/Policy-brief-Crypto-web3-and-the-metaverse.pdf>.
- [16] W. Xu y A. Rudnicky, "Can artificial neural networks learn language models?" En *Proceedings of the 6th International Conference on Spoken Language Processing (ICSLP 2000)*, 2000. DOI: [10.21437/ICSLP.2000-50](https://doi.org/10.21437/ICSLP.2000-50).
- [17] Z. Zhang, Z. Zhang y Z. Wang, "Review of Image Classification Algorithms Based on Convolutional Neural Networks," *Remote Sensing*, 2023. DOI: [10.3390/rs13224712](https://doi.org/10.3390/rs13224712). dirección: <https://doi.org/10.3390/rs13224712>.
- [18] T. Mikolov, K. Chen, G. Corrado y J. Dean, "Efficient Estimation of Word Representations in Vector Space," *arXiv preprint arXiv:1301.3781*, 2013. dirección: <https://doi.org/10.48550/arXiv.1301.3781>.
- [19] I. Sutskever, O. Vinyals y Q. V. Le, "Sequence to Sequence Learning with Neural Networks," *arXiv preprint arXiv:1409.3215*, 2014. dirección: <https://doi.org/10.48550/arXiv.1409.3215>.
- [20] A. Vaswani, N. Shazeer, N. Parmar et al., "Attention Is All You Need," *arXiv preprint arXiv:1706.03762*, 2017. DOI: [10.48550/arXiv.1706.03762](https://doi.org/10.48550/arXiv.1706.03762). dirección: <https://arxiv.org/abs/1706.03762>.
- [21] Google Developers, *LLMs: ¿Qué es un modelo de lenguaje grande?* Dirección: <https://developers.google.com/machine-learning/crash-course/llm/transformers?hl=es-419>.
- [22] J. Uszkoreit, *Transformer: A Novel Neural Network Architecture for Language Understanding*, 2017. dirección: <https://research.google/blog/transformer-a-novel-neural-network-architecture-for-language-understanding/>.

- [23] J. Devlin, M.-W. Chang, K. Lee y K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *arXiv preprint arXiv:1810.04805v2*, 2019. dirección: <https://arxiv.org/abs/1810.04805v2>.
- [24] Aider-AI, *Polyglot Benchmark: Coding problems used in Aider’s polyglot benchmark*. dirección: <https://github.com/Aider-AI/polyglot-benchmark>.
- [25] C. E. Jimenez, J. Yang, A. Wettig et al., “SWE-bench: Can Language Models Resolve Real-world Github Issues?” En *The Twelfth International Conference on Learning Representations*, 2024. dirección: <https://openreview.net/forum?id=VTF8yNQM66>.
- [26] N. Jain, K. Han, A. Gu et al., *LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code*, 2024. dirección: <https://github.com/LiveCodeBench/LiveCodeBench>.
- [27] X. Jiang, L. Wu, S. Sun et al., “Investigating Large Language Models for Code Vulnerability Detection: An Experimental Study,” *arXiv preprint arXiv:2412.18260v2*, 2025. dirección: <https://arxiv.org/abs/2412.18260v2>.
- [28] S. Sultana, S. Afreen y N. U. Eisty, “Code Vulnerability Detection: A Comparative Analysis of Emerging Large Language Models,” *arXiv preprint arXiv:2409.10490v1*, 2024. dirección: <https://arxiv.org/abs/2409.10490>.
- [29] W. Qin, L. Suo, L. Li y F. Yang, “Advancing Software Vulnerability Detection with Reasoning LLMs: DeepSeek-R1’s Performance and Insights,” *Applied Sciences*, 2025. DOI: [10.3390/app15126651](https://doi.org/10.3390/app15126651). dirección: <https://www.mdpi.com/2076-3417/15/12/6651>.
- [30] Solana Developers, *Owner Checks*. dirección: <https://solana.com/es/developers/courses/program-security/owner-checks>.
- [31] Trail of Bits, *Missing Ownership Check*, Not So Smart Contracts – Building Secure Contracts. dirección: https://secure-contracts.com/not-so-smart-contracts/solana/ownership_check/index.html.
- [32] Solana Cookbook, *How to Verify Accounts*. dirección: <https://solanacookbook.com/references/programs.html#how-to-verify-accounts>.

- [33] Solana Developers, *Signer Authorization*. dirección: <https://solana.com/es/developers/courses/program-security/signer-auth>.
- [34] Trail of Bits, *Missing Signer Check*, Not So Smart Contracts – Building Secure Contracts. dirección: https://secure-contracts.com/not-so-smart-contracts/solana/signer_check/index.html.
- [35] anza-xyz, *lib.rs (línea 617) - SDK de Solana v2.2.2*. dirección: <https://github.com/anza-xyz/solana-sdk/blob/sdk@v2.2.2/pubkey/src/lib.rs#L617>.
- [36] The Rust Programming Language, *Método $checked_{add}$ para $i64$* . dirección: https://doc.rust-lang.org/std/primitive.i64.html#method.checked_add.
- [37] The Rust Programming Language, *Método $checked_{sub}$ para $u64$* . dirección: https://doc.rust-lang.org/std/primitive.u64.html#method.checked_sub.
- [38] Solana Developers, *Missing Initialization Check*. dirección: <https://solana.com/es/developers/courses/program-security/reinitialization-attacks#missing-initialization-check>.
- [39] Neodyme, *Solana Account Confusions*, 2021. dirección: https://neodyme.io/en/blog/solana_common_pitfalls/#solana-account-confusions.
- [40] Solana Developers, *Bump Seed Canonicalization*. dirección: <https://solana.com/es/developers/courses/program-security/bump-seed-canonicalization>.
- [41] Trail of Bits, *Improper PDA bump seed validation*, Not So Smart Contracts – Building Secure Contracts. dirección: https://secure-contracts.com/not-so-smart-contracts/solana/improper_pda_validation/index.html.
- [42] Solana Developers, *Arbitrary CPI*. dirección: <https://solana.com/es/developers/courses/program-security/arbitrary-cpi>.
- [43] Trail of Bits, *Arbitrary CPI*. dirección: https://secure-contracts.com/not-so-smart-contracts/solana/arbitrary_cpi/index.html.
- [44] Wormhole, *Introduction to Wormhole*. dirección: <https://wormhole.com/docs/protocol/introduction/>.
- [45] CertiK, *Wormhole Bridge Exploit Incident Analysis*, 2022. dirección: <https://www.certik.com/resources/blog/wormhole-bridge-exploit-incident-analysis>.

- [46] Anza Team, *Documentación del crate solana-secp256k1-program*. dirección: https://docs.rs/solana-secp256k1-program/latest/solana_secp256k1_program/.
- [47] Anza Team, *Documentación de la función `load_instruction_at_solana_program::sysvar::instructions`*, Accedido: 2025-09-10. dirección: https://docs.rs/solana-program/latest/solana_program/sysvar/instructions/fn.load_instruction_at.html.
- [48] Solana Program, *Token-2022*. dirección: <https://www.solana-program.com/docs/token-2022>.
- [49] S. Hassan y P. De Filippi, “Decentralized Autonomous Organization,” *Internet Policy Review*, 2021. DOI: 10.14763/2021.2.1556. dirección: <https://policyreview.info/glossary/DAO>.
- [50] Princeton NLP, *SWE-bench Verified Dataset*, 2024. dirección: https://huggingface.co/datasets/princeton-nlp/SWE-bench_Verified.
- [51] OpenAI, *Introducing SWE-bench Verified*, ago. de 2024. dirección: <https://openai.com/index/introducing-swe-bench-verified/>.
- [52] J. Yang, K. Lieret, C. E. Jimenez et al., *SWE-smith: Scaling Data for Software Engineering Agents*, 2025. arXiv: 2504.21798 [cs.SE]. dirección: <https://arxiv.org/abs/2504.21798>.
- [53] J. Zhou, T. Lu, S. Mishra et al., “Instruction-Following Evaluation for Large Language Models,” *arXiv preprint arXiv:2311.07911*, 2023. dirección: <https://arxiv.org/abs/2311.07911>.
- [54] Y. Wang, X. Ma, G. Zhang et al., “MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark,” *arXiv preprint arXiv:2406.01574*, 2024. dirección: <https://arxiv.org/abs/2406.01574>.
- [55] Y. Wang, X. Ma, G. Zhang et al., *MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark*, 2024. dirección: <https://github.com/TIGER-AI-Lab/MMLU-Pro>.
- [56] OpenAI, *Introducing GPT-4.1 in the API*. dirección: <https://openai.com/index/gpt-4-1/>.

- [57] OpenAI, *GPT-4.1 Nano*, abr. de 2025. dirección: https://www.vals.ai/models/openai_gpt-4-1-nano-2025-04-14.
- [58] Google DeepMind / Google AI, *Gemini 2.0 Flash Model Card*, abr. de 2025. dirección: <https://storage.googleapis.com/model-cards/documents/gemini-2-flash.pdf>.
- [59] DeepSeek, *DeepSeek-V3.1 Release*, ago. de 2025. dirección: <https://api-docs.deepseek.com/news/news250821>.
- [60] A. Yang, A. Li, B. Yang et al., “Qwen3 Technical Report,” *arXiv preprint arXiv:2505.09388*, 2025. dirección: <https://arxiv.org/abs/2505.09388>.
- [61] Qwen Team, *Qwen3: Large Language Model Series*. dirección: <https://github.com/QwenLM/Qwen3>.
- [62] Qwen, *Qwen3-235B-A22B-Instruct-2507*. dirección: <https://huggingface.co/Qwen/Qwen3-235B-A22B-Instruct-2507>.
- [63] P. Lewis, E. Perez, A. Piktus et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *arXiv preprint arXiv:2005.11401v4*, 2021. DOI: [10.48550/arXiv.2005.11401](https://arxiv.org/abs/2005.11401). dirección: <https://arxiv.org/abs/2005.11401>.
- [64] S. Feng, Z. Wang, Y. Wang et al., “Model Swarms: Collaborative Search to Adapt LLM Experts via Swarm Intelligence,” *arXiv preprint arXiv:2410.11163v2*, 2025. dirección: <https://arxiv.org/abs/2410.11163>.



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

E.T.S. DE INGENIERÍA INFORMÁTICA

E.T.S de Ingeniería Informática
Bulevar Louis Pasteur, 35
Campus de Teatinos
29071 Málaga